Contents

May 17, 2026

1 Data Structures

1.1 Dsu

```
/* DSU (Disjoint Set Union / Union-Find)
 * Mantem conjuntos disjuntos com uniao e busca eficiente.
 * Complexidade: O(a(N)) por operacao, onde a e a inversa de Ackermann.
 * Memoria: O(N)
 * Metodos:
 * - find(i): Retorna o representante do conjunto de i.
 * - join(i, j): Une os conjuntos de i e j, retorna true se uniu.
 * - same(i, j): Verifica se i e j estao no mesmo conjunto.
 * - size(i): Retorna o tamanho do conjunto de i.
 */

D56 struct DSU {
1A8     int n;
923     vector<int> parent;
1DF     vector<int> sz;
75E     int num_sets;

DDB     DSU(int _n) : n(_n), parent(_n), sz(_n, 1), num_sets(_n) {
4D6         iota(parent.begin(), parent.end(), 0);
548     }

C42     int find(int i) {
331         if (parent[i] == i) return i;
565         return parent[i] = find(parent[i]);
72C     }

D58     bool join(int i, int j) {
477         i = find(i), j = find(j);
41E         if (i != j) {
4C2             if (sz[i] < sz[j]) swap(i, j);
1F0             parent[j] = i;
529             sz[i] += sz[j];
CA8             num_sets--;
8A6             return true;
20F         }
D1F         return false;
30C     }

00C     int size(int i) { return sz[find(i)]; }
DA3     bool same(int i, int j) { return find(i) == find(j); }
FF6 };
```

1.2 Mo

```
/* Mo's Algorithm — O((N + Q) * sqrt(N))
 * queries: vetor de {l, r} 0-indexado, fechado.
 * add(i), rem(i): adicionam/removem o elemento i da janela.
 * ans(qi): registra resposta para a query de indice qi.
 *
 * Exemplo:
 *   vector<int> ans(q);
 *   Mo mo(n, queries);
 *   mo.run(
 *       [&](int i) { ... },
 *       [&](int i) { ... },
 *       [&](int qi) { ans[qi] = ...; }
 *   );
 */

0DF struct Mo {
7F4     int n, block;
304     struct Query { int l, r, idx; };
240     vector<Query> queries;

8CA     Mo(int n, vector<pair<int,int>>& qs)
10A         : n(n), block(max(1, (int)sqrt(n))) {
D26         for (int i = 0; i < (int)qs.size(); i++)
9AA             queries.push_back({qs[i].first, qs[i].second, i});
F94         sort(queries.begin(), queries.end(),
724             [&](const Query& a, const Query& b) {
ED6                 int ba = a.l / block, bb = b.l / block;
A2D                 if (ba != bb) return ba < bb;
FDF                 return (ba & 1) ? a.r > b.r : a.r < b.r;
FC3             });
8F7     }

D5A     template<typename Add, typename Rem, typename Ans>
743     void run(Add add, Rem rem, Ans ans) {
70A         int L = 0, R = -1;
FFE         for (auto& [l, r, idx] : queries) {
DBE             while (R < r) add(++R);
EBF             while (L > l) add(--L);
153             while (R > r) rem(R--);
319             while (L < l) rem(L++);
414             ans(idx);
4B3         }
1D9     }
09E };

/* HilbertMo — Mo com ordenacao pela curva de Hilbert
 * Mesma interface que Mo, mas com constante menor na pratica.
 * Especialmente util quando N e Q sao grandes e o cache e critico.
 */

CD0 struct HilbertMo {
22A     struct Query { int l, r, idx; long long ord; };
240     vector<Query> queries;

B0A     static long long hilbert(int x, int y, int n) {
CEE         long long d = 0;
F8A         for (int s = n >> 1; s; s >>= 1) {
7B9             int rx = (x & s) > 0, ry = (y & s) > 0;
065             d += (long long)s * s * ((3 * rx) ^ ry);
069             if (!ry) {
1FD                 if (rx) { x = s - 1 - x; y = s - 1 - y; }
9DD                 swap(x, y);
FFF             }
070         }
BE2         return d;
151     }

E76     HilbertMo(int n, vector<pair<int,int>>& qs) {
ABE         int hn = 1;
FCC         while (hn < n) hn <<= 1;
D26         for (int i = 0; i < (int)qs.size(); i++)
F80             queries.push_back({qs[i].first, qs[i].second, i,
C04                                hilbert(qs[i].first, qs[i].second, hn)});
F94         sort(queries.begin(), queries.end(),
B18             [](const Query& a, const Query& b) {
73A                 return a.ord < b.ord;
A3A             });
0D6     }

D5A     template<typename Add, typename Rem, typename Ans>
743     void run(Add add, Rem rem, Ans ans) {
70A         int L = 0, R = -1;
277         for (auto& [l, r, idx, ord] : queries) {
DBE             while (R < r) add(++R);
EBF             while (L > l) add(--L);
153             while (R > r) rem(R--);
319             while (L < l) rem(L++);
```

```
414             ans(idx);
F5F         }
EBB     }
EF4 };
```

1.3 Fenwick Tree

```
/* Fenwick Tree (Point Update, Prefix Query)
 * Realiza atualizacoes pontuais e consultas de prefixo.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(N)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- (para range query) e
 *   construtor default.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

8A0 template<typename NODE>
0F1 struct FenwickTree {
060     int N;
9B8     vector<NODE> bit;

5D2     FenwickTree(int n) : N(n), bit(n + 1) {}

D6B     void update(int idx, NODE v) {
B2F         for (idx++; idx <= N; idx += idx & -idx) {
046             bit[idx] += v;
23E         }
9DA     }

FCA     NODE query(int idx) { // [0, idx]
ADA         NODE res;
159         for (idx++; idx > 0; idx -= idx & -idx) {
8E6             res += bit[idx];
A38         }
B50         return res;
C00     }

762     NODE query(int l, int r) { // [l, r] (precisa ser reversivel)
7C3         if (l > r) return NODE();
A4E         return query(r) - query(l - 1);
821     }
134 };
```

1.4 Fenwick Tree 2D

```
/* Fenwick Tree 2D (Point Update, Rectangle Query)
 * Realiza atualizacoes pontuais e consultas de prefixo em matrizes.
 * Complexidade: O(log N * log M) para update e query.
 * Memoria: O(N * M)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- e construtor default.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

8A0 template<typename NODE>
2E6 struct FenwickTree2D {
610     int N, M;
803     vector<vector<NODE>> bit;

237     FenwickTree2D(int n, int m) : N(n), M(m), bit(n + 1, vector<NODE>(m + 1)) {}

274     void update(int r, int c, NODE v) { // point update
3BA         for (int i = r + 1; i <= N; i += i & -i) {
786             for (int j = c + 1; j <= M; j += j & -j) {
9ED                 bit[i][j] += v;
95F             }
A1F         }
BF9     }

82E     NODE query(int r, int c) { // [(0,0), (r,c)]
ADA         NODE res;
8D9         for (int i = r + 1; i > 0; i -= i & -i) {
83F             for (int j = c + 1; j > 0; j -= j & -j) {
01D                 res += bit[i][j];
CB5             }
E20         }
B50         return res;
943     }

05F     NODE query(int r1, int c1, int r2, int c2) { // [(r1,c1), (r2,c2)]
5C4         if (r1 > r2 || c1 > c2) return NODE();
B7F         return query(r2, c2) - query(r1 - 1, c2)
794                - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
0E7     }
FF7 };
```

1.5 Dynamic Fenwick Tree 2D

```
/* Fenwick Tree 2D Sparse (Point Update, Rectangle Query)
 * Suporta coordenadas ate 10^9 usando compressao interna.
 * Complexidade: Build O(P log P), Update/Query O(log^2 P).
 * Memoria: O(P log P).
 * Requisitos:
 * - NODE deve ter: operator+=, operator-, operator+ e construtor default.
 * - Offline: Todos os pontos de interesse devem ser passados no build.
 *
 * Creditos: Adaptado de TFG (Thiago Ferreira Goncalves)
 * Fonte: github.com/tfg50/Competitive-Programming
 */

/* --- Exemplo de NODE (Soma com Inclusao-Exclusao) ---
struct Node {
    ll val;
    Node(long long v = 0) : val(v) {}

    void operator+=(const Node& other) {
        val += other.val;
    }

    Node operator-(const Node& other) const {
        return Node(val - other.val);
    }

    Node operator+(const Node& other) const {
        return Node(val + other.val);
    }
};
*/

8A0 template<typename NODE>
222 struct FenwickTree2DSparse {
6C1     vector<int> ord;
803     vector<vector<NODE>> bit;
E76     vector<vector<int>> coord;

BEF     FenwickTree2DSparse(vector<pii> pts) {
CD7         sort(pts.begin(), pts.end());
5A8         for (auto [x, y] : pts) {
DC4             if (ord.empty() || x != ord.back()) ord.push_back(x);
ABA         }

261         bit.resize(ord.size() + 1);
3EB         coord.resize(ord.size() + 1);

A0A         sort(pts.begin(), pts.end(), [](const pii& a, const pii& b) {
463             return a.second < b.second;
19C         });

5A8         for (auto [x, y] : pts) {
A1A             for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
3E1                 if (coord[i].empty() || coord[i].back() != y)
739                     coord[i].push_back(y);
D32             }
BAF         }

BE8         for (int i = 0; i < (int)bit.size(); i++) {
F3E             bit[i].assign(coord[i].size() + 1, NODE());
B33         }
84E     }

F1C     inline int get_x(int x) {
2C8         return upper_bound(ord.begin(), ord.end(), x) - ord.begin();
FF7     }

DD7     inline int get_y(int i, int y) {
190         return upper_bound(coord[i].begin(), coord[i].end(), y)
7A2                - coord[i].begin();
B44     }

5CF     void update(int x, int y, NODE v) {
A1A         for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
511             for (int j = get_y(i, y); j < (int)bit[i].size(); j += j & -j) {
9ED                 bit[i][j] += v;
66B             }
638         }
0CE     }

84C     NODE query(int x, int y) {
ADA         NODE res;
FAD         for (int i = get_x(x); i > 0; i -= i & -i) {
263             for (int j = get_y(i, y); j > 0; j -= j & -j) {
01D                 res += bit[i][j];
359             }
92F         }
B50         return res;
7C4     }

FCA     NODE query(int x1, int y1, int x2, int y2) {
CD6         return query(x2, y2) - query(x1 - 1, y2)
CAD                - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
944     }
67D };
```

1.6 Sparse Table

```
/* Sparse Table (Static RMQ)
 * Realiza consultas em range em tempo constante.
 * Complexidade: O(N log N) no build, O(1) na query.
 * Memoria: O(N log N)
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
 * - A operacao DEVE ser idempotente: f(a, a) = a.
 */

/* --- Exemplo de NODE (Indice do Minimo) ---
struct Node {
    int val, pos;
```

```
    Node(int v = 2e9, int p = -1) : val(v), pos(p) {}

    static inline Node merge(const Node& l, const Node& r) {
        return l.val <= r.val ? l : r;
    }
};

// Uso: SparseTable<Node> st(v) onde v e vector<Node>
// for (int i = 0; i < n; i++) { cin >> x; v[i] = Node(x, i); }
*/

template<typename NODE>
struct SparseTable {
    int N, K;
    vector<vector<NODE>> st;
    vector<int> lg;

    template<typename T>
    SparseTable(const vector<T>& v) : N(v.size()) {
        K = (N > 0) ? __lg(N) : 0;
        st.assign(K + 1, vector<NODE>(N));
        lg.assign(N + 1, 0);
        for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] + 1;
        for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);
        for (int j = 1; j <= K; j++)
            for (int i = 0; i + (1 << j) <= N; i++)
                st[j][i] = NODE::merge(st[j - 1][i],
                                       st[j - 1][i + (1 << (j - 1))]);
    }

    NODE query(int l, int r) {
        int j = lg[r - l + 1];
        return NODE::merge(st[j][l], st[j][r - (1 << j) + 1]);
    }
};
```

1.7 Disjoint Sparse Table

```
/* Disjoint Sparse Table (Static Range Query)
 * Realiza consultas em range em O(1) para qualquer operacao associativa.
 * Complexidade: O(N log N) no build, O(1) na query.
 * Memoria: O(N log N)
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
 * - Operacao deve ser ASSOCIATIVA: f(f(a,b),c) = f(a,f(b,c)).
 */

/* --- Exemplo de NODE (Soma em Range) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

template<typename NODE>
struct DisjointSparseTable {
    int N, K;
    vector<vector<NODE>> st;

    template<typename T>
    DisjointSparseTable(const vector<T>& v) : N(v.size()) {
        if (N == 0) return;
        K = __lg(2 * N - 1);
        st.assign(K, vector<NODE>(1 << K));

        for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);

        for (int j = 1; j < K; j++) {
            int len = 1 << j;
            for (int m = len; m <= (1 << K); m += 2 * len) {
                if (m < N) st[j][m] = st[0][m];
                for (int i = m + 1; i < min(N, m + len); i++)
                    st[j][i] = NODE::merge(st[j][i - 1], st[0][i]);

                if (m - 1 < N) st[j][m - 1] = st[0][m - 1];
                for (int i = m - 2; i >= m - len; i--)
                    st[j][i] = NODE::merge(st[0][i], st[j][i + 1]);
            }
        }
    }

    inline NODE query(int l, int r) {
        if (l == r) return st[0][l];
        int j = __lg(l ^ r);
        return NODE::merge(st[j][l], st[j][r]);
    }
};
```

1.8 Segment Tree

```
/* Segment Tree (Point Update, Range Query)
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(V), e construtor identidade.
 */

/* --- Exemplo de NODE (Soma com Update de Substituicao) ---
struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}

    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }

    // Para Point Set: val = v;
    // Para Point Add: val += v;
    inline void apply(int v) {
        val = v;
    }
};
*/

template<typename NODE>
struct SegTree {
    int N;
    vector<NODE> seg;

    SegTree(int n) : N(n), seg(4 * n) {}

    template<typename T>
    SegTree(const vector<T>& v) : N(v.size()), seg(4 * v.size()) {
        build(1, 0, N - 1, v);
    }

    template<typename T>
    void build(int no, int l, int r, const vector<T>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
        }
        int m = (l + r) >> 1;
        build(no << 1, l, m, v);
        build((no << 1) | 1, m + 1, r, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    void update(int no, int l, int r, int idx, int val) {
        if (l == r) {
            seg[no].apply(val);
            return;
        }
        int m = (l + r) >> 1;
        if (idx <= m) update(no << 1, l, m, idx, val);
        else update((no << 1) | 1, m + 1, r, idx, val);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a, b));
    }

    void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};
```

1.9 Lazy Segment Tree

```
/* Lazy Segment Tree (Range Update, Range Query)
 * Realiza atualizacoes e consultas em range.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
 *   construtor identidade.
 * - TAG deve ter: compose(TAG) e construtor identidade.
 */

/* --- Exemplo de NODE e TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void inline compose(const Tag& t) {
        add = (add * t.mul + t.add) % MOD;
        mul = (mul * t.mul) % MOD;
    }
};

struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node((l.val + r.val) % MOD);
    }
    void inline apply(const Tag& t, int l, int r) {
        val = (val * t.mul + t.add * (r - l + 1)) % MOD;
    }
};
*/

template<typename NODE, typename TAG>
struct LazySegmentTree {
    int N;
    vector<NODE> seg;
    vector<TAG> lazy;

    LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}

    template<typename T>
    LazySegmentTree(const vector<T>& v)
        : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
        build(1, 0, N - 1, v);
    }

    template<typename T>
    void build(int no, int l, int r, const vector<T>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
```

```
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);
C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349     }

F08     void push(int no, int l, int r) {
27E         int m = (l + r) >> 1;
1EA         int e = no << 1, d = e | 1;

25F         seg[e].apply(lazy[no], l, m);
115         lazy[e].compose(lazy[no]);

4E1         seg[d].apply(lazy[no], m + 1, r);
7CC         lazy[d].compose(lazy[no]);

47F         lazy[no] = TAG();
6A3     }

130     void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6         if (b < l || r < a) return;
02B         if (a <= l && r <= b) {
8C7             seg[no].apply(v, l, r);
92C             lazy[no].compose(v);
505             return;
81F         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
A1A     }

38D     NODE query(int no, int l, int r, int a, int b) {
2E6         if (b < l || r < a) return NODE();
83F         if (a <= l && r <= b) return seg[no];
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
AFE         return NODE::merge(query(no << 1, l, m, a, b),
074                            query((no << 1) | 1, m + 1, r, a, b));
80A     }

0D7     void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
36F     NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
54E };
```

1.10 Dual Segment Tree

```
/* Dual Segment Tree (Range Update, Point Query)
 * Realiza atualizacoes em range e consultas pontuais.
 * Complexidade: O(log N) para update e get.
 * Memoria: O(4*N)
 * Requisitos:
 * - TAG deve ter: compose(TAG), apply(T&) e construtor identidade.
 * - T e o tipo do dado nas folhas.
 * Exemplo de uso: DualSegTree<ll, MyTag> st(v);
 */

/* --- Exemplo de TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void compose(const Tag& t) {
        mul = (mul * t.mul) % MOD;
        add = (add * t.mul + t.add) % MOD;
    }
    void apply(ll& val) { val = (val * mul + add) % MOD; }
};
*/

2E5 template<typename T, typename TAG>
9C1 struct DualSegTree {
060     int N;
71A     vector<TAG> lazy;
E4B     vector<T> leaves;

11D     DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
5F6     DualSegTree(const vector<T>& v)
62C         : N(v.size()), lazy(4 * v.size()), leaves(v) {}

F08     void push(int no, int l, int r) {
F7F         int e = no << 1;
EE1         int d = e | 1;
115         lazy[e].compose(lazy[no]);
7CC         lazy[d].compose(lazy[no]);
47F         lazy[no] = TAG();
48C     }

130     void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6         if (b < l || r < a) return;
02B         if (a <= l && r <= b) {
92C             lazy[no].compose(v);
505             return;
CA0         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
667     }

629     T get(int no, int l, int r, int idx) {
893         if (l == r) {
4C5             T res = leaves[idx];
2E3             lazy[no].apply(res);
B50             return res;
BE7         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
52E         if (idx <= m) return get(no << 1, l, m, idx);
483         else return get((no << 1) | 1, m + 1, r, idx);
A24     }

A21     void update(int l, int r, const TAG& t) { update(1, 0, N - 1, l, r, t); }
AE4     T get(int idx) { return get(1, 0, N - 1, idx); }
6C2 };
```

1.11 Segment Tree Beats

```
/* Segment Tree Beats (Range Update, Range Query)
 * Extensao da Lazy Seg Tree com condicoes de parada e de tag.
 * Complexidade amortizada: O(N log^2 N) para sequencias de updates.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(TAG, L, R),
 *   break_condition(TAG) e tag_condition(TAG),
 *   e construtor identidade.
 * - TAG deve ter: compose(TAG) e construtor identidade.
 *
 * break_condition(tag): retorna true se o update pode ser ignorado
 *                       (o no ja satisfaz a condicao — ex: mx <= v).
 * tag_condition(tag):   retorna true se a tag pode ser propagada
 *                       diretamente sem descer (ex: mx2 < v <= mx).
 */

/* --- Exemplo de NODE e TAG (chmin em range + query de soma/max) ---
struct Tag {
    ll val = LLONG_MAX; // identidade: nao faz nada
    void inline compose(const Tag& t) {
        val = min(val, t.val);
    }
};

struct Node {
    ll sum, mx, mx2;
    int cnt_mx, sz;

    Node() : sum(0), mx(LLONG_MIN), mx2(LLONG_MIN), cnt_mx(0), sz(0) {}
    Node(ll v) : sum(v), mx(v), mx2(LLONG_MIN), cnt_mx(1), sz(1) {}

    static inline Node merge(const Node& l, const Node& r) {
        Node res;
        res.sz = l.sz + r.sz;
        res.sum = l.sum + r.sum;
        if (l.mx == r.mx) {
            res.mx = l.mx; res.cnt_mx = l.cnt_mx + r.cnt_mx;
            res.mx2 = max(l.mx2, r.mx2);
        } else if (l.mx > r.mx) {
            res.mx = l.mx; res.cnt_mx = l.cnt_mx;
            res.mx2 = max(l.mx2, r.mx);
        } else {
            res.mx = r.mx; res.cnt_mx = r.cnt_mx;
            res.mx2 = max(l.mx, r.mx2);
        }
        return res;
    }

    // aplica a tag quando tag_condition e verdadeiro
    void inline apply(const Tag& t, int l, int r) {
        sum -= (ll)(mx - t.val) * cnt_mx;
        mx = t.val;
    }

    // ignora o update completamente (no ja satisfaz)
    bool inline break_condition(const Tag& t) const {
        return mx <= t.val;
    }

    // pode aplicar a tag diretamente sem descer
    bool inline tag_condition(const Tag& t) const {
        return mx2 < t.val;
    }
};
*/

708 template<typename NODE, typename TAG>
BC8 struct SegTreeBeats {
060     int N;
B1C     vector<NODE> seg;
71A     vector<TAG> lazy;

8EA     SegTreeBeats(int n) : N(n), seg(4 * n), lazy(4 * n) {}

67A     template<typename T>
47B     SegTreeBeats(const vector<T>& v)
9EB         : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
231         build(1, 0, N - 1, v);
5EE     }

67A     template<typename T>
5B9     void build(int no, int l, int r, const vector<T>& v) {
893         if (l == r) {
C3E             seg[no] = NODE(v[l]);
505             return;
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);
C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349     }

F08     void push(int no, int l, int r) {
27E         int m = (l + r) >> 1;
1EA         int e = no << 1, d = e | 1;

25F         seg[e].apply(lazy[no], l, m);
115         lazy[e].compose(lazy[no]);

4E1         seg[d].apply(lazy[no], m + 1, r);
```

```
        lazy[d].compose(lazy[no]);

        lazy[no] = TAG();
    }

    void update(int no, int l, int r, int a, int b, const TAG& v) {
        if (b < l || r < a || seg[no].break_condition(v)) return;
        if (a <= l && r <= b && seg[no].tag_condition(v)) {
            seg[no].apply(v, l, r);
            lazy[no].compose(v);
            return;
        }
        push(no, l, r);
        int m = (l + r) >> 1;
        update(no << 1, l, m, a, b, v);
        update((no << 1) | 1, m + 1, r, a, b, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        push(no, l, r);
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a, b));
    }

    void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};
```

1.12 Dynamic Segment Tree

```
/* Dynamic Segment Tree with Lazy Propagation (Range Update, Range Query)
 * Utilizada para intervalos gigantes (ex: 0 a 10^18).
 * Complexidade: O(log N) por operacao.
 * Memoria: O(Q log N) onde Q e o numero de operacoes.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&),
 *   apply(const TAG&, ll, ll) e construtor identidade.
 * - TAG deve ter: apply(const TAG&), campo 'has' (bool) e
 *   construtor identidade.
 */

/* --- Exemplo de NODE e TAG ---
struct Tag {
    ll add = 0;
    bool has = false;
    void apply(const Tag& t) {
        add += t.add;
        has = true;
    }
};

struct Node {
    ll val = 0;
    static Node merge(const Node& l, const Node& r) {
        return {l.val + r.val};
    }
    void apply(const Tag& t, ll l, ll r) {
        val += t.add * (r - l + 1);
    }
};
*/

template<typename NODE, typename TAG>
struct DynamicSegTree {
    struct InternalNode {
        NODE node;
        TAG tag;
        int l, r;

        InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}

        void apply(const TAG& t, ll l, ll r) {
            node.apply(t, l, r);
            tag.apply(t);
        }
    };

    ll L, R;
    vector<InternalNode> seg;

    DynamicSegTree(ll l, ll r) : L(l), R(r) {
        seg.reserve(4e6); seg.emplace_back();
    }

    void push(int v, ll l, ll r) {
        if (!seg[v].tag.has) return;
        if (l == r) return seg[v].tag = TAG();

        if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }

        ll mid = l + (r - l) / 2;
        seg[seg[v].l].apply(seg[v].tag, l, mid);
        seg[seg[v].r].apply(seg[v].tag, mid+1, r);

        seg[v].tag = TAG();
    }

    void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) {
        if (ql <= l && r <= qr) return seg[v].apply(t, l, r);

        push(v, l, r);
        ll mid = l + (r - l) / 2;
        if (ql <= mid) {
            if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
            update(seg[v].l, l, mid, ql, qr, t);
        }
        if (qr > mid) {
            if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
            update(seg[v].r, mid + 1, r, ql, qr, t);
        }

        NODE left = (seg[v].l != -1) ? seg[seg[v].l].node : NODE();
        NODE rght = (seg[v].r != -1) ? seg[seg[v].r].node : NODE();
        seg[v].node = NODE::merge(left, rght);
    }

    NODE query(int v, ll l, ll r, ll ql, ll qr) {
        if (v == -1 || ql > r || qr < l) return NODE();
        if (ql <= l && r <= qr) return seg[v].node;
        push(v, l, r);
        ll mid = l + (r - l) / 2;
        return NODE::merge(query(seg[v].l, l, mid, ql, qr),
                           query(seg[v].r, mid + 1, r, ql, qr));
    }

    void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
    NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
};
```

1.13 Dynamic Persistent Segment Tree

```
/* Persistent Dynamic Segment Tree (Point Update, Range Query)
 * Permite consultas em versoes anteriores da arvore (persistencia).
 * Aloca nos sob demanda, ideal para intervalos grandes e multiplas versoes.
 * Complexidade: O(log N) por update e query.
 * Memoria: O(Q log N) onde Q e o numero de updates.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e
 *   construtor identidade.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    int val;
    Node(int v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

template<typename NODE>
struct PersistentSegmentTree {
    struct InternalNode {
        NODE data;
        int l, r;
        InternalNode() : data(NODE()), l(0), r(0) {}
    };

    int N;
    vector<InternalNode> st;
    vector<int> roots;

    PersistentSegmentTree(int n) : N(n) {
        st.reserve(4e6); st.emplace_back();
        roots.push_back(0);
    }

    int update(int prev_root, int L, int R, int idx, NODE v) {
        int node = st.size();
        if (prev_root == 0) st.emplace_back();
        else st.push_back({st[prev_root]});

        if (L == R) {
            st[node].data = NODE::merge(st[node].data, v);
            return node;
        }

        int mid = L + (R - L) / 2;

        if (idx <= mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
            int prev_left = (prev_root == 0) ? 0 : st[prev_root].l;
            st[node].l = update(prev_left, L, mid, idx, v);
        }

        else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
            int prev_right = (prev_root == 0) ? 0 : st[prev_root].r;
            st[node].r = update(prev_right, mid + 1, R, idx, v);
        }

        st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
        return node;
    }

    NODE query(int node, int L, int R, int i, int j) {
        if (node == 0 || i > R || j < L) return NODE();
        if (i <= L && R <= j) return st[node].data;
        int mid = L + (R - L) / 2;
        return NODE::merge(query(st[node].l, L, mid, i, j),
                           query(st[node].r, mid + 1, R, i, j));
    }

    void update(int idx, NODE v) {
        roots.push_back(update(roots.back(), 0, N - 1, idx, v));
    }

    NODE query(int version, int l, int r) {
        return query(roots[version], 0, N - 1, l, r);
    }
};
```

2 Graphs

2.1 Dijkstra

```
/* Dijkstra — Caminho minimo de fonte unica
 * Complexidade: O((V + E) log V)
 * Requer pesos nao-negativos.
 *
 * adj[u] = lista de {v, w}: aresta u→v com peso w
 *
 * Retorna vetor dist onde dist[v] = distancia minima de s a v.
 * dist[v] = INF se v e inalcancavel.
 */

template<typename T>
vector<T> dijkstra(const vector<vector<pair<int,T>>>& adj, int s) {
    int n = adj.size();
    const T INF = numeric_limits<T>::max();
    vector<T> dist(n, INF);
    priority_queue<pair<T,int>, vector<pair<T,int>>, greater<>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
    }
    return dist;
}
```

2.2 Euler Path

```
/* Caminho/Circuito Euleriano — Hierholzer — O(V + E)
 *
 * Condicoes de existencia:
 *   Grafo nao-direcionado:
 *     - Circuito: todos os vertices com grau par.
 *     - Caminho:  exatamente 2 vertices com grau impar (inicio e fim).
 *   Grafo direcionado:
 *     - Circuito: in_degree[v] == out_degree[v] para todo v.
 *     - Caminho:  exatamente 1 vertice com out-in=+1 (inicio),
 *                 exatamente 1 com in-out=+1 (fim), resto iguais.
 *
 * euler(adj, s) → vetor de vertices do caminho (tamanho E+1).
 *   Retorna vetor vazio se nao existe caminho euleriano a partir de s.
 *   adj[u] = lista de {v, edge_id}: aresta u→v com indice edge_id.
 *   Para grafo nao-direcionado, adicione cada aresta em ambas as direcoes
 *   com o mesmo edge_id; o algoritmo marca arestas visitadas pelo id.
 */

vector<int> euler(vector<vector<pair<int,int>>>& adj, int s) {
    int E = 0;
    for (auto& v : adj) E += v.size();
    E /= 2; // para nao-direcionado; para direcionado, remova o /2

    vector<bool> used_edge(E, false);
    vector<int> ptr(adj.size(), 0);
    vector<int> path, stk = {s};

    while (!stk.empty()) {
        int v = stk.back();
        bool found = false;
        while (ptr[v] < (int)adj[v].size()) {
            auto [u, eid] = adj[v][ptr[v]++];
            if (!used_edge[eid]) {
                used_edge[eid] = true;
                stk.push_back(u);
                found = true;
                break;
            }
        }
        if (!found) {
            path.push_back(v);
            stk.pop_back();
        }
    }

    reverse(path.begin(), path.end());
    return path;
}
```

2.3 Lca

```
#include "../data-structures/sparse-table/sparse-table.hpp"

/* LCA (Lowest Common Ancestor) via RMQ
 * Encontra o ancestral comum mais proximo de dois vertices em uma arvore.
 * Complexidade: O(N log N) no build, O(1) por query.
 * Memoria: O(N log N)
 * Requisitos:
 * - Grafo deve ser uma arvore (conexo e aciclico).
 * - Dependencia: sparse-table/sparse-table.hpp
 * Metodos:
 * - lca(u, v): Retorna o LCA de u e v.
 * - dist(u, v): Retorna a distancia entre u e v.
 * - is_ancestor(u, v): Verifica se u e ancestral de v.
 * - parent(u): Retorna o pai de u.
 */

struct LCANode {
    int dep, id;
    LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos) {}

    static inline LCANode merge(const LCANode& l, const LCANode& r) {
        return l.dep < r.dep ? l : r;
    }
};

struct LCA {
    int N;
    vector<int> first, tour, d, p;
    SparseTable<LCANode> st;

    LCA(int n, int root, const vector<vector<int>>& adj)
        : N(n), first(n), d(n), p(n, -1), st(vector<LCANode>()) {

        vector<int> tour_depths;
        tour.reserve(2 * n);
        tour_depths.reserve(2 * n);

        auto dfs = [&](auto self, int u, int parent_id, int dep) -> void {
            p[u] = parent_id;
            d[u] = dep;
            first[u] = tour.size();
            tour.push_back(u);
            tour_depths.push_back(dep);
            for (int v : adj[u]) {
                if (v == parent_id) continue;
                self(self, v, u, dep + 1);
                tour.push_back(u);
                tour_depths.push_back(dep);
            }
        };

        dfs(dfs, root, -1, 0);
        st = SparseTable<LCANode>(tour_depths);
    }

    int parent(int u) {
        return p[u];
    }

    int lca(int u, int v) {
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        return tour[st.query(l, r).id];
    }

    int dist(int u, int v) {
        return d[u] + d[v] - 2 * d[lca(u, v)];
    }

    bool is_ancestor(int u, int v) {
        return lca(u, v) == u;
    }
};
```

2.4 Tarjan

```
/* Tarjan SCC (Strongly Connected Components)
 * Complexidade: O(V + E)
 *
 * Campos publicos apos build():
 *   comp[v]  = id do SCC de v (0-indexado, ordem topologica reversa)
 *   scc[i]   = lista de vertices do i-esimo SCC
 *   dag      = grafo condensado (arestas entre SCCs distintos, sem duplicatas)
 *   n_scc    = numero de SCCs
 *
 * Nota: comp[] esta em ordem topologica reversa do DAG condensado,
 *       i.e., se ha aresta SCC a → SCC b, entao comp[a] > comp[b].
 *       Para ordem topologica direta, percorra de n_scc-1 a 0.
 */

struct Tarjan {
    int n, n_scc = 0;
    vector<vector<int>> g, scc, dag;
    vector<int> comp;

    Tarjan(int n) : n(n), g(n), comp(n, -1) {}

    Tarjan(const vector<vector<int>>& adj)
        : n(adj.size()), g(adj), comp(adj.size(), -1) { build(); }

    void add_edge(int u, int v) {
        g[u].push_back(v);
    }

    void build() {
        vector<int> low(n), num(n, -1), stk;
        vector<bool> on_stk(n, false);
        int timer = 0;

        function<void(int)> dfs = [&](int v) {
            low[v] = num[v] = timer++;
            stk.push_back(v);
            on_stk[v] = true;
            for (int u : g[v]) {
                if (num[u] == -1) { dfs(u); low[v] = min(low[v], low[u]); }
                else if (on_stk[u]) low[v] = min(low[v], num[u]);
            }
            if (low[v] == num[v]) {
                scc.push_back({});
                while (true) {
                    int u = stk.back(); stk.pop_back();
                    on_stk[u] = false;
                    comp[u] = n_scc;
                    scc.back().push_back(u);
                    if (u == v) break;
                }
                n_scc++;
```

```
6CD             }
5A5         };

830         for (int i = 0; i < n; i++)
917             if (num[i] == -1) dfs(i);

9E4         dag.assign(n_scc, {});
19E         for (int u = 0; u < n; u++)
4FB             for (int v : g[u])
C76                 if (comp[u] != comp[v])
FC7                     dag[comp[u]].push_back(comp[v]);

404         for (auto& adj : dag) {
324             sort(adj.begin(), adj.end());
942             adj.erase(unique(adj.begin(), adj.end()), adj.end());
80E         }
BA2     }
6D7 };
```

2.5 Articulations

```
/* Pontos de Articulacao — Tarjan
 * Complexidade: O(V + E)
 * Grafo nao-direcionado.
 *
 * Um vertice e ponto de articulacao se sua remocao desconecta o grafo.
 *
 * Campos publicos apos build():
 *   is_ap[v] = true se v e ponto de articulacao
 *   get()    = lista de vertices que sao pontos de articulacao
 */

E5A struct Articulations {
1A8     int n;
789     vector<vector<int>> g;
BC9     vector<bool> is_ap;

620     Articulations(int n) : n(n), g(n), is_ap(n, false) {}

6F5     Articulations(const vector<vector<int>>& adj)
B9D         : n(adj.size()), g(adj), is_ap(adj.size(), false) { build(); }

58B     void add_edge(int u, int v) {
F2B         g[u].push_back(v);
4D6         g[v].push_back(u);
FFF     }

0A8     void build() {
AC3         vector<int> num(n, -1), low(n);
2DC         int timer = 0;

CF3         function<void(int,int)> dfs = [&](int v, int par) {
006             low[v] = num[v] = timer++;
E07             int children = 0;
E32             for (int u : g[v]) {
403                 if (num[u] == -1) {
C5F                     children++;
412                     dfs(u, v);
C80                     low[v] = min(low[v], low[u]);
369                     if (par == -1 && children > 1) is_ap[v] = true;
1B3                     if (par != -1 && low[u] >= num[v]) is_ap[v] = true;
709                 } else if (u != par) {
2D3                     low[v] = min(low[v], num[u]);
933                 }
9F1             }
8CA         };

830         for (int i = 0; i < n; i++)
10B             if (num[i] == -1) dfs(i, -1);
DEE     }

    // Retorna lista de vertices que sao pontos de articulacao
D3D     vector<int> get() const {
02F         vector<int> res;
830         for (int i = 0; i < n; i++)
BA3             if (is_ap[i]) res.push_back(i);
B50         return res;
60A     }
A5D };
```

2.6 Bridges

```
/* Pontes (Bridges) — Tarjan
 * Complexidade: O(V + E)
 * Grafo nao-direcionado.
 *
 * Uma aresta e ponte se sua remocao desconecta o grafo.
 *
 * Campos publicos apos build():
 *   bridges = lista de arestas {u, v} que sao pontes
 */

D44 struct Bridges {
1A8     int n;
789     vector<vector<int>> g;
0CC     vector<pair<int,int>> bridges;

B0A     Bridges(int n) : n(n), g(n) {}

C65     Bridges(const vector<vector<int>>& adj)
5F7         : n(adj.size()), g(adj) { build(); }

58B     void add_edge(int u, int v) {
F2B         g[u].push_back(v);
4D6         g[v].push_back(u);
FFF     }

0A8     void build() {
AC3         vector<int> num(n, -1), low(n);
2DC         int timer = 0;

CF3         function<void(int,int)> dfs = [&](int v, int par) {
006             low[v] = num[v] = timer++;
E32             for (int u : g[v]) {
403                 if (num[u] == -1) {
412                     dfs(u, v);
C80                     low[v] = min(low[v], low[u]);
2FE                     if (low[u] > num[v]) bridges.push_back({v, u});
FB4                 } else if (u != par) {
2D3                     low[v] = min(low[v], num[u]);
E66                 }
7A3             }
938         };

830         for (int i = 0; i < n; i++)
10B             if (num[i] == -1) dfs(i, -1);
A6F     }
000 };
```

2.7 2-SAT

```
C31 #include "tarjan.hpp"

/* 2-SAT — Complexidade: O(V + E)
 *
 * Literal x: use i para x_i=true, ~i para x_i=false.
 *
 *   add_impl(x, y) →  x → y  (contrapositiva ~y → ~x automatica)
 *   add_or(x, y) →    x v y
 *   add_xor(x, y) →   exatamente um de x, y e true
 *   add_iff(x, y) →   x ↔ y
 *   force(x) →        x = true
 *   solve() →         {satisfativel, ans[]}; ans[i] = 0 ou 1
 */

D9D struct TwoSat {
1A8     int n;
0D7     Tarjan t;
DAB     vector<int> ans;

46B     TwoSat(int n) : n(n), t(2 * n), ans(n, -1) {}

    // converte literal para no interno: i >= 0 → 2*i, i < 0 → -2*i-1
5D7     static int node(int x) { return x >= 0 ? 2 * x : -2 * x - 1; }

    // implicacao x → y (e contrapositiva ~y → ~x)
74A     void add_impl(int x, int y) {
D64         int nx = node(x), ny = node(y);
2B2         t.add_edge(nx, ny);
378         t.add_edge(ny ^ 1, nx ^ 1);
255     }

    // x v y
F10     void add_or(int x, int y) { add_impl(~x, y); }

    // x XOR y (exatamente um verdadeiro)
C02     void add_xor(int x, int y) { add_or(x, y); add_or(~x, ~y); }

    // x ↔ y
721     void add_iff(int x, int y) { add_impl(x, y); add_impl(~x, ~y); }

    // forca x = true
5F8     void force(int x) { add_impl(~x, x); }

A8E     pair<bool, vector<int>> solve() {
2BD         t.build();
E1C         ans.assign(n, -1);
603         for (int i = 0; i < n; i++) {
EBA             if (t.comp[2 * i] == t.comp[2 * i + 1]) return {false, {}};
7B4             ans[i] = t.comp[2 * i] > t.comp[2 * i + 1] ? 1 : 0;
291         }
997         return {true, ans};
BB1     }
F3D };
```

2.8 Hld Commutative

```
6A0 #include "../../data-structures/segment-tree/segment-tree.hpp"

/* HLD Comutativo (Update Pontual)
 * Para operacoes onde A + B = B + A.
 * Requisito: data-structures/segment-tree/segment-tree.hpp
 */

8A0 template<typename NODE>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
115     SegTree<NODE> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833         : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

898         vector<vector<int>> g = adj;
227         d[root] = 0, p[root] = root;

94B         auto dfs_sz = [&](auto self, int u) -> void {
267             sz[u] = 1;
839             int best_v = -1, max_sz = -1;
A8C             for (auto &v : g[u]) {
567                 if (v == p[u]) continue;
ABE                 d[v] = d[u] + 1; p[v] = u;
033                 self(self, v);
CC3                 sz[u] += sz[v];
```

```
                if (sz[v] > max_sz) {
                    max_sz = sz[v];
                    best_v = v;
                }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<NODE> base(n);
        for(int i = 0; i < n; i++)
            base[pos[i]] = vals[i];
        st = SegTree<NODE>(base);
    }

    void update(int u, const NODE& val) {
        st.update(pos[u], val);
    }

    NODE query_path(int u, int v) {
        NODE res;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        return NODE::merge(res, st.query(pos[u], pos[v]));
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1);
    }
};
```

2.9 Hld Commutative Lazy

```
#include "../../data-structures/segment-tree/lazy-segment-tree.hpp"

/* HLD Comutativo com Lazy Propagation
 * Para operacoes onde A + B = B + A.
 * Permite updates e queries em caminhos e subarvores.
 * Requisito: data-structures/segment-tree/lazy-segment-tree.hpp
 */

template<typename NODE, typename TAG>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos;
    LazySegmentTree<NODE, TAG> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
        : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<NODE> base(n);
        for(int i = 0; i < n; i++)
            base[pos[i]] = vals[i];
        st = LazySegmentTree<NODE, TAG>(base);
    }

    void update_path(int u, int v, const TAG& tag) {
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            st.update(pos[head[v]], pos[v], tag);
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        st.update(pos[u], pos[v], tag);
    }

    NODE query_path(int u, int v) {
        NODE res;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        return NODE::merge(res, st.query(pos[u], pos[v]));
    }

    void update_subtree(int u, const TAG& tag) {
        st.update(pos[u], pos[u] + sz[u] - 1, tag);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1);
    }
};
```

2.10 Hld Non Commutative

```
#include "../../data-structures/segment-tree/segment-tree.hpp"

/* HLD Nao-Comutativo (Update Pontual)
 * Para operacoes onde A * B != B * A.
 * Requisito: DoubleNode para gerenciar merges em ambas as direcoes.
 */

template<typename NODE>
struct DoubleNode {
    NODE down, up;
    DoubleNode() : down(NODE()), up(NODE()) {}
    DoubleNode(const NODE& n) : down(n), up(n) {}
    DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

    static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
        return {
            NODE::merge(l.down, r.down),
            NODE::merge(r.up, l.up)
        };
    }
};

template<typename NODE>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos, tour;
    SegTree<DoubleNode<NODE>> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), tour(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            tour[pos[u]] = u;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<DoubleNode<NODE>> base(n);
        for(int i = 0; i < n; i++)
```

```
            base[pos[i]] = DoubleNode<NODE>(vals[i]);
        st = SegTree<DoubleNode<NODE>>(base);
    }

    void update(int u, const NODE& val) {
        st.update(pos[u], DoubleNode<NODE>(val));
    }

    NODE query_path(int u, int v) {
        NODE L, R;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
                u = p[head[u]];
            } else {
                R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) L = NODE::merge(L, st.query(pos[v], pos[u]).up);
        else R = NODE::merge(st.query(pos[u], pos[v]).down, R);

        return NODE::merge(L, R);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1).down;
    }
};
```

2.11 Hld Non Commutative Lazy

```
#include "../../data-structures/segment-tree/lazy-segment-tree.hpp"

/* HLD Nao-Comutativo com Lazy Propagation
 * Para operacoes onde A + B != B + A (ex: Matrizes, Funcoes Lineares).
 * Requisito: DoubleNode que guarda versoes 'normal' e 'invertida' do merge.
 */

template<typename NODE>
struct DoubleNode {
    NODE down, up;

    DoubleNode() : down(NODE()), up(NODE()) {}
    DoubleNode(const NODE& n) : down(n), up(n) {}
    DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

    static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
        return {
            NODE::merge(l.down, r.down),
            NODE::merge(r.up, l.up)
        };
    }

    template<typename TAG>
    void apply(const TAG& t, int l, int r) {
        down.apply(t, l, r);
        up.apply(t, l, r);
    }
};

template<typename NODE, typename TAG>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos;
    LazySegmentTree<DoubleNode<NODE>, TAG> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
        : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<DoubleNode<NODE>> base(n);
        for(int i = 0; i < n; i++) base[pos[i]] = DoubleNode<NODE>(vals[i]);
        st = LazySegmentTree<DoubleNode<NODE>, TAG>(base);
    }

    void update_path(int u, int v, const TAG& tag) {
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                st.update(pos[head[u]], pos[u], tag);
                u = p[head[u]];
            } else {
                st.update(pos[head[v]], pos[v], tag);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) st.update(pos[v], pos[u], tag);
        else st.update(pos[u], pos[v], tag);
    }

    NODE query_path(int u, int v) {
        NODE L, R;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
                u = p[head[u]];
            } else {
                R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) {
            L = NODE::merge(L, st.query(pos[v], pos[u]).up);
        } else {
            R = NODE::merge(st.query(pos[u], pos[v]).down, R);
        }
        return NODE::merge(L, R);
    }

    void update_subtree(int u, const TAG& tag) {
        st.update(pos[u], pos[u] + sz[u] - 1, tag);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1).down;
    }
};
```

2.12 Dinic

```
/* Dinic (Max Flow)
 * Complexidade: O(V^2 * E) geral, O(E * sqrt(V)) em grafos unitarios.
 * Memoria: O(V + E)
 *
 * Metodos:
 *   add_edge(u, v, cap) ->        aresta u->v com capacidade cap
 *   add_edge(u, v, cap, rcap) ->  idem com aresta reversa rcap (nao-direcionado)
 *   flow(s, t) ->                 fluxo maximo de s a t
 *   min_cut(s) ->  apos flow(), vetor onde [i]=true se i esta no lado da fonte
 */

template<typename T>
struct Dinic {
    struct Edge {
        int to, rev;
        T cap;
    };

    int N;
    vector<vector<Edge>> g;
    vector<int> level, iter;

    Dinic(int n) : N(n), g(n), level(n), iter(n) {}

    void add_edge(int u, int v, T cap, T rcap = 0) {
        g[u].push_back({v, (int)g[v].size(), cap});
        g[v].push_back({u, (int)g[u].size() - 1, rcap});
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto& e : g[v])
                if (e.cap > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
        }
        return level[t] >= 0;
    }

    T dfs(int v, int t, T f) {
        if (v == t) return f;
        for (int& i = iter[v]; i < (int)g[v].size(); i++) {
            Edge& e = g[v][i];
            if (e.cap > 0 && level[v] < level[e.to]) {
                T d = dfs(e.to, t, min(f, e.cap));
                if (d > 0) {
                    e.cap -= d;
                    g[e.to][e.rev].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }

    T flow(int s, int t) {
```

```
        T res = 0;
        while (bfs(s, t)) {
            fill(iter.begin(), iter.end(), 0);
            T d;
            while ((d = dfs(s, t, numeric_limits<T>::max())) > 0)
                res += d;
        }
        return res;
    }

    vector<bool> min_cut(int s) {
        vector<bool> vis(N, false);
        queue<int> q;
        vis[s] = true;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto& e : g[v])
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = true;
                    q.push(e.to);
                }
        }
        return vis;
    }
};
```

2.13 MCMF

```
/* MCMF — Min-Cost Max-Flow (SPFA/Bellman-Ford based)
 * Complexidade: O(V * E * flow) no pior caso; na pratica muito mais rapido.
 * Memoria: O(V + E)
 *
 * Metodos:
 *   add_edge(u, v, cap, cost) →       aresta →uv com cap e custo
 *   add_edge(u, v, cap, rcap, cost) → idem com aresta reversa rcap
 *   flow(s, t) →                      {fluxo maximo, custo minimo} de s a t
 *   flow(s, t, max_flow) →            idem limitando o fluxo total a max_flow
 *
 * Observacoes:
 *   - Suporta custos negativos (usa SPFA em vez de Dijkstra).
 *   - Sem custos negativos, prefira Dijkstra+potenciais: O(E log V)/aug.
 *   - O custo retornado e o custo minimo para enviar o fluxo maximo.
 */

template<typename Cap, typename Cost>
struct MCMF {
    struct Edge {
        int to, rev;
        Cap cap;
        Cost cost;
    };

    int N;
    vector<vector<Edge>> g;

    MCMF(int n) : N(n), g(n) {}

    void add_edge(int u, int v, Cap cap, Cost cost, Cap rcap = 0) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, rcap, -cost});
    }

    pair<Cap, Cost> flow(int s, int t,
                         Cap max_flow = numeric_limits<Cap>::max()) {
        Cap total_flow = 0;
        Cost total_cost = 0;

        while (max_flow > 0) {
            vector<Cost> dist(N, numeric_limits<Cost>::max());
            vector<bool> in_queue(N, false);
            vector<int> prev_node(N, -1), prev_edge(N, -1);

            dist[s] = 0;
            queue<int> q;
            q.push(s);
            in_queue[s] = true;

            while (!q.empty()) {
                int v = q.front(); q.pop();
                in_queue[v] = false;
                for (int i = 0; i < (int)g[v].size(); i++) {
                    auto& e = g[v][i];
                    if (e.cap > 0 && dist[v] + e.cost < dist[e.to]) {
                        dist[e.to] = dist[v] + e.cost;
                        prev_node[e.to] = v;
                        prev_edge[e.to] = i;
                        if (!in_queue[e.to]) {
                            in_queue[e.to] = true;
                            q.push(e.to);
                        }
                    }
                }
            }

            if (dist[t] == numeric_limits<Cost>::max()) break;

            Cap pushed = max_flow;
            for (int v = t; v != s; v = prev_node[v])
                pushed = min(pushed, g[prev_node[v]][prev_edge[v]].cap);

            for (int v = t; v != s; v = prev_node[v]) {
                auto& e = g[prev_node[v]][prev_edge[v]];
                e.cap -= pushed;
                g[v][e.rev].cap += pushed;
            }

            total_flow += pushed;
            total_cost += pushed * dist[t];
            max_flow -= pushed;
        }

        return {total_flow, total_cost};
    }
};
```

2.14 Bipartite Matching

```
#include "dinic.hpp"

/* Bipartite Matching via Dinic
 * Matching maximo em grafo bipartido.
 * Complexidade: O(E * sqrt(V))
 *
 * Vertices esquerda: [0, n), direita: [n, n+m).
 *
 * Campos publicos apos match():
 *   size   = tamanho do matching maximo
 *   match_l[i] = vertice da direita matched com i (-1 se nao matched)
 *   match_r[j] = vertice da esquerda matched com j (-1 se nao matched)
 *
 * Metodos:
 *   add_edge(i, j) → aresta entre esquerda i e direita j
 *   match() →        executa o matching, retorna tamanho
 */

struct BipartiteMatching {
    int n, m;
    Dinic<int> g;
    int S, T;
    vector<int> match_l, match_r;
    int size = 0;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + m + 2),
          S(n + m), T(n + m + 1),
          match_l(n, -1), match_r(m, -1) {
        for (int i = 0; i < n; i++) g.add_edge(S, i, 1);
        for (int j = 0; j < m; j++) g.add_edge(n + j, T, 1);
    }

    void add_edge(int i, int j) {
        g.add_edge(i, n + j, 1);
    }

    int match() {
        size = g.flow(S, T);
        for (int i = 0; i < n; i++)
            for (auto& e : g.g[i])
                if (e.to != S && e.cap == 0) {
                    match_l[i] = e.to - n;
                    match_r[e.to - n] = i;
                }
        return size;
    }
};
```

3 Math

3.1 Mint

```
/* mint — Inteiro modular
 * Complexidade: O(1) por operacao, O(log mod) para inv()
 *
 * template<typename T, T mod>
 *
 * Uso comum:
 *   using mint = Mint<int, 998244353>;
 *   using mll  = Mint<ll, 998244353LL>;
 *
 * Operacoes: +, -, *, /, ==, !=, <<
 * inv() → inverso modular (mod deve ser primo)
 */

template<typename T, T mod>
struct Mint {
    T v;
    Mint(T v = 0) : v(((v % mod) + mod) % mod) {}

    Mint operator+(Mint o) const { return v+o.v >= mod ? v+o.v-mod : v+o.v; }
    Mint operator-(Mint o) const { return v-o.v < 0   ? v-o.v+mod : v-o.v; }
    Mint operator*(Mint o) const { return (ll)v * o.v % mod; }
    Mint operator/(Mint o) const { return *this * o.inv(); }

    Mint& operator+=(Mint o) { return *this = *this + o; }
    Mint& operator-=(Mint o) { return *this = *this - o; }
    Mint& operator*=(Mint o) { return *this = *this * o; }
    Mint& operator/=(Mint o) { return *this = *this / o; }

    bool operator==(Mint o) const { return v == o.v; }
    bool operator!=(Mint o) const { return v != o.v; }

    explicit operator T() const { return v; }

    Mint inv() const {
        T a = v, b = mod, x = 1, y = 0;
        while (b) { T q = a/b; a -= q*b; swap(a,b); x -= q*y; swap(x,y); }
        return x;
    }

    friend ostream& operator<<(ostream& os, Mint m) { return os << m.v; }
};
```

3.2 Fast Exponentiation

```
/* fexp — Exponenciacao rapida
 * Complexidade: O(log b)
 *
 * fexp(a, b) → a^b  usando operator* do tipo T
 *
 * Use com mint/mll para exponenciacao modular automatica.
 *
 * modinv<mod>(a) → a^{-1} % mod  (mod deve ser primo)
 */

67A template<typename T>
83D T fexp(T a, ll b) {
EC7     T res = T(1);
0EB     for (; b > 0; b >>= 1, a = a*a)
FC2         if (b & 1) res = res * a;
B50     return res;
0CE }

31E template<ll mod>
611 ll modinv(ll a) { return fexp<ll>(a % mod, mod - 2); }
```

3.3 Sieve

```
/* Crivo de Eratostenes
 * Complexidade: O(N log log N)
 *
 * sieve(n) → vetor com todos os primos ate n (inclusive).
 */

705 vector<int> sieve(int n) {
EFA     vector<bool> is_prime(n + 1, true);
FD3     vector<int> primes;
19E     is_prime[0] = is_prime[1] = false;
8A9     for (int p = 2; p <= n; p++) {
29C         if (is_prime[p]) {
BCE             primes.push_back(p);
908             if ((long long)p * p <= n)
4C7                 for (int i = p * p; i <= n; i += p)
F10                     is_prime[i] = false;
FFE         }
5D1     }
A20     return primes;
81C }
```

3.4 Number Theory

```
/* GCD / LCM — O(log min(a, b)) */

225 ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
15C ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }

/* Divisores de n — O(sqrt(N))
 * Retorna lista de divisores em ordem crescente.
 */

5FA vector<ll> divisors(ll n) {
17E     vector<ll> lo, hi;
148     for (ll i = 1; i * i <= n; i++) {
C06         if (n % i == 0) {
1EB             lo.push_back(i);
9C3             if (i != n / i) hi.push_back(n / i);
B5B         }
2DE     }
FC1     reverse(hi.begin(), hi.end());
3AF     lo.insert(lo.end(), hi.begin(), hi.end());
253     return lo;
B9C }

/* Fatoracao em primos — O(sqrt(N))
 * Retorna lista de {primo, expoente} em ordem crescente.
 */

7E3 vector<pair<ll,int>> factorize(ll n) {
618     vector<pair<ll,int>> factors;
D2E     for (ll p = 2; p * p <= n; p++) {
80A         if (n % p == 0) {
A01             int exp = 0;
6C9             while (n % p == 0) { n /= p; exp++; }
13A             factors.push_back({p, exp});
2E9         }
47D     }
992     if (n > 1) factors.push_back({n, 1});
FA9     return factors;
92A }

/* CRT — Teorema Chines dos Restos — O(log min(m1, m2))
 * Retorna {x, lcm(m1,m2)} tal que x ≡ a1 (mod m1) e x ≡ a2 (mod m2).
 * Retorna {0, 0} se nao ha solucao (m1, m2 nao coprimos e incompativeis).
 */

8A9 pair<ll,ll> crt(ll a1, ll m1, ll a2, ll m2) {
8F7     ll g = gcd(m1, m2);
916     if ((a2 - a1) % g != 0) return {0, 0};
9C3     ll l = m1 / g * m2;
D0B     ll a = m1/g, b = m2/g, x = 1, y = 0;
E58     for (ll ta = a, tb = b; tb; ) {
146         ll q = ta/tb; ta -= q*tb; swap(ta,tb); x -= q*y; swap(x,y);
54E     }
581     x = (x % (m2/g) + m2/g) % (m2/g);
EE5     ll r = (a1 + m1 * ((a2 - a1) / g % (m2/g) * x % (m2/g))) % l;
D27     return {(r + l) % l, l};
F7F }

/* Fatoracao usando lista de primos pre-computada — Oπ((sqrt(N)))
 * Use quando ja tiver rodado sieve(sqrt(n)) antes de chamar para varios n.
 * primes deve conter todos os primos ate sqrt(n).
 */

E6D vector<pair<ll,int>> factorize(ll n, const vector<int>& primes) {
618     vector<pair<ll,int>> factors;
3B9     for (int p : primes) {
851         if ((ll)p * p > n) break;
80A         if (n % p == 0) {
A01             int exp = 0;
6C9             while (n % p == 0) { n /= p; exp++; }
13A             factors.push_back({p, exp});
2E9         }
17C     }
992     if (n > 1) factors.push_back({n, 1});
FA9     return factors;
490 }
```

3.5 Miller Rabin

```
/* Miller-Rabin + Pollard Rho
 *
 * is_prime(n) →  O(log² n), deterministico para n < 3.3⁸·10¹
 * factorize(n) → O(n^{1/4} log n) esperado; fatores com repeticao
 *
 * Use sort + unique para fatores distintos, ou agrupe por expoente.
 */

7D4 ll mulmod(ll a, ll b, ll m) { return (__int128)a * b % m; }

0A0 bool is_prime(ll n) {
5A7     if (n < 2) return false;
0A8     if (n == 2 || n == 3 || n == 5 || n == 7) return true;
3CE     if (n % 2 == 0 || n % 3 == 0) return false;
B87     ll d = n - 1; int r = 0;
5B7     while (d % 2 == 0) d /= 2, r++;
C12     for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
D22         if (a >= n) continue
8C8         ll x = 1, base = a % n, exp = d;
006         for (; exp > 0; exp >>= 1, base = mulmod(base, base, n))
5E6             if (exp & 1) x = mulmod(x, base, n);
8A3         if (x == 1 || x == n - 1) continue;
30D         bool composite = true;
21B         for (int i = 0; i < r - 1; i++) {
F38             x = mulmod(x, x, n);
789             if (x == n - 1) { composite = false; break; }
4C7         }
4F2         if (composite) return false;
D5C     }
8A6     return true;
E78 }

711 ll pollard_rho(ll n) {
55A     if (n % 2 == 0) return 2;
2BC     ll x = rand() % (n - 2) + 2, y = x, c = rand() % (n - 1) + 1, d = 1;
4EB     while (d == 1) {
C17         x = (mulmod(x, x, n) + c) % n;
4EE         y = (mulmod(y, y, n) + c) % n;
4EE         y = (mulmod(y, y, n) + c) % n;
DD8         d = __gcd(abs(x - y), n);
565     }
9EC     return d == n ? pollard_rho(n) : d;
5D7 }

6C2 vector<ll> factorize(ll n) {
1B9     if (n == 1) return {};
970     if (is_prime(n)) return {n};
B32     ll d = pollard_rho(n);
F9F     auto a = factorize(d), b = factorize(n / d);
19E     a.insert(a.end(), b.begin(), b.end());
3F5     return a;
E13 }
```

3.6 Matrix

```
AD1 #include "fexp.hpp"

/* Matrix — Matriz quadrada com multiplicacao e exponenciacao rapida
 * Complexidade: O(n³ log k) para M^k
 *
 * Use com mint/mll para operacoes modulares automaticas.
 *
 * Matrix<T> M(n) →   identidade n×n
 * Matrix<T> M(n,0) → matriz n×n zerada
 * M[i][j] →          acesso direto
 * A * B →            multiplicacao O(n³)
 * M ^ k →            M^k usando fexp
 */

67A template<typename T>
BF3 struct Matrix {
1A8     int n;
C24     vector<vector<T>> mat;

C62     Matrix(int n, int identity = 1) : n(n), mat(n, vector<T>(n, T(0))) {
F03         if (identity) for (int i = 0; i < n; i++) mat[i][i] = T(1);
6BA     }

90D     vector<T>& operator[](int i) { return mat[i]; }
699     const vector<T>& operator[](int i) const { return mat[i]; }

AFB     Matrix operator*(const Matrix& rhs) const {
83B         Matrix res(n, 0);
830         for (int i = 0; i < n; i++)
23D             for (int k = 0; k < n; k++) if (mat[i][k] != T(0))
F90                 for (int j = 0; j < n; j++)
C55                     res[i][j] = res[i][j] + mat[i][k] * rhs.mat[k][j];
B50         return res;
```

```
893     }

6FC     Matrix operator^(long long b) const { return fexp(*this, b); }
732 };
```

3.7 Gaussian Elimination

```
 /* Eliminacao Gaussiana (RREF) — O(N^2 * M)
  * Funciona com double ou Mint<T, mod>.
  *
  * gauss(A, b) → resolve Ax = b.
  *   Retorna {x, true} se unica, {x, false} se inconsistente/sub-determinado.
  */

67A template<typename T>
5D7 pair<vector<T>, bool> gauss(vector<vector<T>> A, vector<T> b) {
FE3     int n = A.size();
3E0     for (int i = 0; i < n; i++) A[i].push_back(b[i]);
3D7     int m = n + 1, rank = 0;
079     for (int col = 0; col < n && rank < n; col++) {
9D3         int sel = rank;
F97         for (int i = rank + 1; i < n; i++)
E75             if (abs(A[i][col]) > abs(A[sel][col])) sel = i;
9E6         if (!(bool)A[sel][col]) continue;
D8D         swap(A[sel], A[rank]);
B46         T iv = T(1) / A[rank][col];
C24         for (int j = col; j < m; j++) A[rank][j] = A[rank][j] * iv;
603         for (int i = 0; i < n; i++) {
BC7             if (i == rank || !(bool)A[i][col]) continue;
BEF             T f = A[i][col];
BAC             for (int j = col; j < m; j++) A[i][j] = A[i][j] - f * A[rank][j];
371         }
56E         rank++;
3F2     }
175     if (rank < n) return {{}, false};
A1B     vector<T> x(n);
B95     for (int i = 0; i < n; i++) x[i] = A[i].back();
079     return {x, true};
724 }
```

3.8 Linear Basis

```
 /* Linear Basis (Base Linear XOR)
  * Complexidade: O(B) por operacao, onde B = numero de bits (64 para ll)
  * Memoria: O(B)
  *
  * Metodos:
  *   insert(x) →     insere x; retorna true se x e linearmente independente
  *   contains(x) →   true se x e XOR de algum subconjunto da base
  *   max_xor(x=0) → maior XOR de x com qualquer subconjunto da base
  *   min_xor() →     menor XOR nao-nulo possivel (0 se base incompleta)
  *   size() →        numero de elementos linearmente independentes
  *   merge(other) → une duas bases (insere todos os elementos de other)
  *
  * Observacoes:
  *   - Base em forma reduzida (cada bit pivot aparece em apenas uma linha).
  *   - Para k-esimo XOR crescente, use reduce() e trate como numero binario.
  */

538 template<typename T = ll, int B = 63>
F45 struct LinearBasis {
E9C     array<T, B + 1> basis{};
1FC     int sz = 0;

220     bool insert(T x) {
C8C         for (int i = B; i >= 0; i--) {
760             if (!((x >> i) & 1)) continue;
617             if (!basis[i]) {
C6C                 basis[i] = x;
EB9                 sz++;
8A6                 return true;
50C             }
6B0             x ^= basis[i];
8A2         }
D1F         return false;
CD7     }

394     bool contains(T x) const {
C8C         for (int i = B; i >= 0; i--) {
760             if (!((x >> i) & 1)) continue;
260             if (!basis[i]) return false;
6B0             x ^= basis[i];
1A9         }
8A6         return true;
189     }

C3B     T max_xor(T x = 0) const {
086         for (int i = B; i >= 0; i--)
EC9             x = max(x, x ^ basis[i]);
EA5         return x;
74B     }

BBB     T min_xor() const {
F56         for (int i = 0; i <= B; i++)
15D             if (basis[i]) return basis[i];
BB3         return 0;
5FC     }

21A     int size() const { return sz; }

AC0     void merge(const LinearBasis& other) {
086         for (int i = B; i >= 0; i--)
D67             if (other.basis[i]) insert(other.basis[i]);
1CA     }

    /* Reduz a base para forma escalonada reduzida (cada pivot tem 1 bit).
     * Necessario para consultas de k-esimo XOR.
     * Apos reduce(), qualquer subconjunto gera um XOR distinto:
     * e possivel enumerar todos os 2^sz XORs possiveis.
     */
93A     void reduce() {
C8C         for (int i = B; i >= 0; i--) {
F14             if (!basis[i]) continue;
E36             for (int j = i + 1; j <= B; j++)
7E7                 if ((basis[j] >> i) & 1) basis[j] ^= basis[i];
CA5         }
563     }

    /* k-esimo menor XOR (0-indexado) entre todos os 2^sz subconjuntos.
     * Requer reduce() antes. Se sz < B+1, XOR=0 = subconjunto vazio.
     * k=0 → 0; k=1 → menor XOR nao-nulo, etc.
     */
0B4     T kth(ll k) const {
E55         vector<T> elems;
F56         for (int i = 0; i <= B; i++)
210             if (basis[i]) elems.push_back(basis[i]);
        // elems esta em ordem crescente de bit mais significativo
D55         if (k >= (1LL << (int)elems.size())) return -1; // k fora do range
19A         T res = 0;
687         for (int i = 0; i < (int)elems.size(); i++)
A7C             if ((k >> i) & 1) res ^= elems[i];
B50         return res;
B1A     }
DE8 };
```

3.9 Unimodal Search

```
 /* Busca unimodal — f deve ser unimodal no intervalo dado.
  *
  * ternary_min/max(lo, hi, f) →          inteiros, O(log N)
  * golden_min/max(lo, hi, f, iters=200) → continuo, O(iters); 200 iters ≈ 1e-30
  */

 // Ternary search para minimo em inteiros [lo, hi]
398 template<typename F>
209 long long ternary_min(long long lo, long long hi, F f) {
960     while (hi - lo > 2) {
C9D         long long m1 = lo + (hi - lo) / 3;
A33         long long m2 = hi - (hi - lo) / 3;
2EE         if (f(m1) <= f(m2)) hi = m2;
F67         else                lo = m1;
885     }
52F     long long best = lo;
35B     for (long long x = lo + 1; x <= hi; x++)
EB8         if (f(x) < f(best)) best = x;
F26     return best;
DB5 }

 // Ternary search para maximo em inteiros [lo, hi]
398 template<typename F>
E4E long long ternary_max(long long lo, long long hi, F f) {
960     while (hi - lo > 2) {
C9D         long long m1 = lo + (hi - lo) / 3;
A33         long long m2 = hi - (hi - lo) / 3;
750         if (f(m1) >= f(m2)) hi = m2;
F67         else                lo = m1;
344     }
52F     long long best = lo;
35B     for (long long x = lo + 1; x <= hi; x++)
592         if (f(x) > f(best)) best = x;
F26     return best;
2AD }

 // Golden section search para minimo em continuo [lo, hi]
398 template<typename F>
938 double golden_min(double lo, double hi, F f, int iters = 200) {
84E     const double phi = (sqrt(5.0) - 1.0) / 2.0;  // ≈ 0.618
0DD     double m1 = hi - phi * (hi - lo);
86C     double m2 = lo + phi * (hi - lo);
87F     double f1 = f(m1), f2 = f(m2);
5B6     for (int i = 0; i < iters; i++) {
F4D         if (f1 < f2) {
D90             hi = m2; m2 = m1; f2 = f1;
251             m1 = hi - phi * (hi - lo); f1 = f(m1);
735         } else {
B6E             lo = m1; m1 = m2; f1 = f2;
9FA             m2 = lo + phi * (hi - lo); f2 = f(m2);
E4A         }
DC3     }
E67     return (lo + hi) / 2.0;
2D2 }

 // Golden section search para maximo em continuo [lo, hi]
398 template<typename F>
CEA double golden_max(double lo, double hi, F f, int iters = 200) {
8ED     return golden_min(lo, hi, [&](double x) { return -f(x); }, iters);
2E7 }
```

3.10 FFT

```
658 using ld = double; // troque por long double para mais precisao
316 using CD = complex<ld>;

 /* FFT — Fast Fourier Transform
  * Complexidade: O(N log N)
  *
  * fft(a) →       transforma a[] no lugar (tamanho deve ser potencia de 2)
  * conv(a, b) →   convolucao: c[k] = Σ a[i]*b[k-i]
  *
  * Precisao: seguro se Σ(a²Σ+b²)*₂logN < 9⁴·10¹.
  * Para coeficientes grandes ou mod, use fft-mod.hpp ou ntt.hpp.
  */

B4C void fft(vector<CD>& a) {
```

```
    int n = a.size(), L = 31 - __builtin_clz(n);

    static vector<complex<long double>> R(2, 1);
    static vector<CD> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        auto x = polar(1.0L, acos(-1.0L) / k);
        R.resize(n); rt.resize(n);
        for (int i = k; i < 2*k; i++)
            rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }

    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i/2] | (i&1) << L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);

    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2*k)
            for (int j = 0; j < k; j++) {
                auto x = (ld*)&rt[j+k], y = (ld*)&a[i+j+k];
                CD z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
                a[i+j+k] = a[i+j] - z;
                a[i+j] += z;
            }
}

vector<ld> conv(const vector<ld>& a, const vector<ld>& b) {
    if (a.empty() || b.empty()) return {};
    vector<ld> res(a.size() + b.size() - 1);
    int n = 1 << (32 - __builtin_clz(res.size()));
    vector<CD> in(n), out(n);
    copy(begin(a), end(a), begin(in));
    for (int i = 0; i < (int)b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (auto& x : in) x *= x;
    for (int i = 0; i < n; i++) out[i] = in[-i&(n-1)] - conj(in[i]);
    fft(out);
    for (int i = 0; i < (int)res.size(); i++) res[i] = imag(out[i]) / (4*n);
    return res;
}
```

3.11 FFT Mod

```
#include "fft.hpp"

/* FFT Mod — Convolucao modular com mod arbitrario
 * Complexidade: O(N log N) (~2x mais lento que FFT ou NTT)
 *
 * convMod<mod>(a, b) → c[k] = Σ( a[i]*b[k-i]) % mod
 *
 * Entradas devem estar em [0, mod).
 * Seguro se 2N·logN·mod < 8.64·10^1.
 *
 * Alta precisao sem mod: troque cut = 1<<15 e remova os % mod.
 */

template<int mod>
vector<long long> convMod(
    const vector<long long>& a, const vector<long long>& b) {
    if (a.empty() || b.empty()) return {};
    vector<long long> res(a.size() + b.size() - 1);
    int B = 32 - __builtin_clz(res.size()), n = 1 << B;
    int cut = (int)sqrt(mod); // alta precisao: use 1<<15 e tire os % mod abaixo
    vector<CD> L(n), R(n), outs(n), outl(n);
    for (int i = 0; i < (int)a.size(); i++)
        L[i] = CD((int)a[i] / cut, (int)a[i] % cut);
    for (int i = 0; i < (int)b.size(); i++)
        R[i] = CD((int)b[i] / cut, (int)b[i] % cut);
    fft(L); fft(R);
    for (int i = 0; i < n; i++) {
        int j = -i & (n-1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / CD(0, 1);
    }
    fft(outl); fft(outs);
    for (int i = 0; i < (int)res.size(); i++) {
        // alta precisao: tire os % mod nas linhas abaixo
        long long av = (long long)(real(outl[i]) + .5) % mod;
        long long bv = (long long)(imag(outl[i]) + .5)
                     + (long long)(real(outs[i]) + .5);
        long long cv = (long long)(imag(outs[i]) + .5);
        res[i] = ((av * cut + bv) % mod * cut + cv) % mod;
    }
    return res;
}
```

3.12 NTT

```
/* NTT — Number Theoretic Transform
 * Complexidade: O(N log N)
 *
 * NTT<mod, g>::conv(a, b) → c[k] = Σ( a[i]*b[k-i]) % mod
 *
 * Entradas devem estar em [0, mod).
 * mod deve ser primo da forma 2^a·b+1 (garante raiz de unidade de ordem 2^a).
 * g e a raiz primitiva de mod — nao e escolha livre, e fixo para cada mod.
 *
 * Opcoes de {mod, g}:
 *   {998244353,          3} → 119·2^23+1,          mais comum,  N ≤ 2^23
 *   {469762049,          3} → 7^6·2^2+1,           N ≤ 2^6^2
 *   {167772161,          3} → 5^5·2^2+1,           N ≤ 2^5^2
 *   {2013265921,        31} → 15^7·2^2+1,          N ≤ 2^7^2   (mod > 9^10)
 *   {2281701377,         3} → 17^7·2^2+1,          N ≤ 2^7^2   (mod > 9^10)
 *   {7340033,            5} → 7^0·2^2+1,           N ≤ 2^0^2   (menor mod)
 *   {9223372036737335297LL,3} → 549755813881^4·2^2+1, ≤^4N^2 (mod^6~2^3, ll)
 */

template<ll mod, ll g>
struct NTT {
    static ll fexp(ll a, ll b) {
        ll res = 1; a %= mod;
        for (; b > 0; b >>= 1, a = a*a%mod)
            if (b & 1) res = res*a%mod;
        return res;
    }

    static void ntt(vector<ll>& a) {
        int n = a.size(), L = 31 - __builtin_clz(n);
        static vector<ll> rt(2, 1);
        for (static int k = 2, s = 2; k < n; k *= 2, s++) {
            rt.resize(n);
            ll z[] = {1, fexp(g, mod >> s)};
            for (int i = k; i < 2*k; i++) rt[i] = rt[i/2] * z[i&1] % mod;
        }
        vector<int> rev(n);
        for (int i = 0; i < n; i++) rev[i] = (rev[i/2] | (i&1) << L) / 2;
        for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
        for (int k = 1; k < n; k *= 2)
            for (int i = 0; i < n; i += 2*k)
                for (int j = 0; j < k; j++) {
                    ll z = rt[j+k] * a[i+j+k] % mod, &ai = a[i+j];
                    a[i+j+k] = ai - z + (z > ai ? mod : 0);
                    ai += z - (ai+z >= mod ? mod : 0);
                }
    }

    static vector<ll> conv(vector<ll> a, vector<ll> b) {
        if (a.empty() || b.empty()) return {};
        int s = a.size()+b.size()-1, n = 1 << (32 - __builtin_clz(s));
        a.resize(n); b.resize(n);
        ntt(a); ntt(b);
        ll inv = fexp(n, mod-2);
        for (int i = 0; i < n; i++) b[-i&(n-1)] = a[i]*b[i]%mod*inv%mod;
        ntt(b);
        return {b.begin(), b.begin()+s};
    }
};
```

3.13 FWHT

```
/* FWHT — Fast Walsh-Hadamard Transform
 * Complexidade: O(N log N)
 *
 * Convolucao bitwise: c[k] = Σ a[i]*b[j] onde i op j = k
 *
 * fwht_conv<'^'>(a, b) → convolucao XOR
 * fwht_conv<'|'>(a, b) → convolucao OR
 * fwht_conv<'&'>(a, b) → convolucao AND
 *
 * Tamanhos sao ajustados para a proxima potencia de 2 automaticamente.
 */

template<char op>
void fwht(vector<ll>& a, bool inv = false) {
    int n = a.size();
    for (int len = 1; len < n; len *= 2)
        for (int i = 0; i < n; i += 2*len)
            for (int j = 0; j < len; j++) {
                ll u = a[i+j], v = a[i+j+len];
                if (op == '^') a[i+j] = u+v, a[i+j+len] = u-v;
                if (op == '|') a[i+j+len] = v + (inv ? -u : +u);
                if (op == '&') a[i+j]     = u + (inv ? -v : +v);
            }
    if (op == '^' && inv) for (auto& x : a) x /= n;
}

template<char op>
vector<ll> fwht_conv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n *= 2;
    a.resize(n); b.resize(n);
    fwht<op>(a); fwht<op>(b);
    for (int i = 0; i < n; i++) a[i] *= b[i];
    fwht<op>(a, true);
    return a;
}
```

4 String

4.1 KMP

```
/* KMP (Knuth-Morris-Pratt)
 * failure[i] = tamanho do maior prefixo proprio de s[0..i] que e tambem sufixo.
 * Complexidade: O(N) build, O(N + M) busca.
 *
 * Aplicacoes:
 * - Busca de padrao P em texto T em O(|P| + |T|).
 * - Menor periodo da string: N - failure[N-1].
 * - Contar ocorrencias de todos os prefixos como sufixos.
 */

vector<int> kmp_failure(const string& s) {
    int n = s.size();
    vector<int> f(n, 0);
    for (int i = 1; i < n; i++) {
        int j = f[i - 1];
        while (j > 0 && s[i] != s[j]) j = f[j - 1];
        if (s[i] == s[j]) j++;
        f[i] = j;
    }
    return f;
}
```

```cpp
// Busca todas as ocorrencias de pattern em text. Retorna indices (0-indexed).
vector<int> kmp_search(const string& pattern, const string& text) {
    string s = pattern + "$" + text;
    vector<int> f = kmp_failure(s);
    int p = pattern.size();
    vector<int> res;
    for (int i = p + 1; i < (int)s.size(); i++)
        if (f[i] == p) res.push_back(i - 2 * p - 1);
    return res;
}
```

4.2 Z Function

```cpp
/* Z-Function
 * z[i] = tamanho do maior prefixo de s que coincide com s[i..].
 * z[0] = 0 por convencao.
 * Complexidade: O(N) build.
 *
 * Aplicacoes:
 * - Busca de padrao P em texto T: z-function de P + "$" + T.
 *   Posicoes i onde z[i] == |P| sao ocorrencias de P em T.
 * - Contar ocorrencias de prefixo de tamanho k: contar z[i] >= k.
 * - Menor periodo da string: menor p tal que p divide N e z[p] == N - p.
 */

vector<int> z_function(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}

// Busca todas as ocorrencias de pattern em text. Retorna indices (0-indexed).
vector<int> z_search(const string& pattern, const string& text) {
    string s = pattern + "$" + text;
    vector<int> z = z_function(s);
    int p = pattern.size();
    vector<int> res;
    for (int i = p + 1; i < (int)s.size(); i++)
        if (z[i] == p) res.push_back(i - p - 1);
    return res;
}
```

4.3 String Hash

```cpp
/* String Hashing (Single e Double Hash)
 * Hash polinomial para comparacao de substrings em O(1) apos build O(N).
 * Complexidade: O(N) build, O(1) query.
 * Memoria: O(N)
 *
 * SingleHash: um par (MOD, BASE) — risco baixo de colisao para uso geral.
 * DoubleHash: dois pares independentes — colisao praticamente impossivel.
 *
 * Uso:
 *   SingleHash h(s);
 *   h.get(l, r); // hash de s[l..r]
 *   h.get(l, r) == h.get(l2, r2); // compara substrings
 */

using ull = unsigned long long;

/* --- Single Hash --- */
struct SingleHash {
    static const ll MOD = (1LL << 61) - 1;
    static const ll BASE;
    vector<ll> h, pw;

    static ll mul(ll a, ll b) {
        return (__uint128_t)a * b % MOD;
    }

    static ll add(ll a, ll b) {
        a += b;
        return a >= MOD ? a - MOD : a;
    }

    SingleHash() {}

    SingleHash(const string& s) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
        for (int i = 0; i < (int)s.size(); i++) {
            h[i + 1] = add(mul(h[i], BASE), s[i]);
            pw[i + 1] = mul(pw[i], BASE);
        }
    }

    ll get(int l, int r) const {
        return add(h[r + 1], MOD - mul(h[l], pw[r - l + 1]));
    }
};

const ll SingleHash::BASE = []() {
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    return uniform_int_distribution<ll>(1e9, SingleHash::MOD - 1)(rng);
}();

/* --- Double Hash --- */
struct DoubleHash {
    static const ll MOD1 = (1LL << 61) - 1;
    static const ll MOD2 = 1e9 + 9;
    static const ll BASE1, BASE2;

    vector<ll> h1, h2, pw1, pw2;

    static ll mul1(ll a, ll b) { return (__uint128_t)a * b % MOD1; }
    static ll add1(ll a, ll b) { a += b; return a >= MOD1 ? a - MOD1 : a; }
    static ll mul2(ll a, ll b) { return (__int128)a * b % MOD2; }
    static ll add2(ll a, ll b) { a += b; return a >= MOD2 ? a - MOD2 : a; }

    DoubleHash() {}

    DoubleHash(const string& s)
        : h1(s.size() + 1, 0), h2(s.size() + 1, 0),
          pw1(s.size() + 1, 1), pw2(s.size() + 1, 1) {
        for (int i = 0; i < (int)s.size(); i++) {
            h1[i + 1] = add1(mul1(h1[i], BASE1), s[i]);
            h2[i + 1] = add2(mul2(h2[i], BASE2), s[i]);
            pw1[i + 1] = mul1(pw1[i], BASE1);
            pw2[i + 1] = mul2(pw2[i], BASE2);
        }
    }

    // retorna par de hashes de s[l..r] (0-indexed, inclusive)
    pair<ll, ll> get(int l, int r) const {
        ll v1 = add1(h1[r + 1], MOD1 - mul1(h1[l], pw1[r - l + 1]));
        ll v2 = add2(h2[r + 1], MOD2 - mul2(h2[l], pw2[r - l + 1]));
        return {v1, v2};
    }
};

const ll DoubleHash::BASE1 = []() {
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    return uniform_int_distribution<ll>(1e9, DoubleHash::MOD1 - 1)(rng);
}();

const ll DoubleHash::BASE2 = []() {
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() + 1);
    return uniform_int_distribution<ll>(1e9, DoubleHash::MOD2 - 1)(rng);
}();
```

4.4 Trie

```cpp
/* Trie (Prefix Tree)
 * Insercao e busca de strings em O(|s|).
 * Memoria: O(N * ALPHA) onde N e o total de caracteres inseridos.
 * Requisitos: strings sobre o alfabeto [base, base + ALPHA).
 */

template<int ALPHA = 26, char BASE = 'a'>
struct Trie {
    struct Node {
        int ch[ALPHA];
        int cnt;
        int end;
        Node() : cnt(0), end(0) { fill(ch, ch + ALPHA, -1); }
    };

    vector<Node> t;

    Trie() { t.emplace_back(); }

    void insert(const string& s) {
        int no = 0;
        for (char c : s) {
            int b = c - BASE;
            if (t[no].ch[b] == -1) {
                t[no].ch[b] = t.size();
                t.emplace_back();
            }
            no = t[no].ch[b];
            t[no].cnt++;
        }
        t[no].end++;
    }

    bool search(const string& s) const {
        int no = 0;
        for (char c : s) {
            int b = c - BASE;
            if (t[no].ch[b] == -1) return false;
            no = t[no].ch[b];
        }
        return t[no].end > 0;
    }

    int count_prefix(const string& s) const {
        int no = 0;
        for (char c : s) {
            int b = c - BASE;
            if (t[no].ch[b] == -1) return 0;
            no = t[no].ch[b];
        }
        return t[no].cnt;
    }

    int count(const string& s) const {
        int no = 0;
        for (char c : s) {
            int b = c - BASE;
            if (t[no].ch[b] == -1) return 0;
            no = t[no].ch[b];
        }
        return t[no].end;
    }
};
```

4.5 Manacher

```cpp
/* Manacher's Algorithm (Palindrome Detection)
 * Encontra todos os palindromos em uma string em tempo linear.
```

```
 * Complexidade: O(N) onde N e o tamanho da string.
 * Memoria: O(N)
 * Funcoes:
 * - manacher(s): arr[i] = tamanho do maior palindromo centrado em i.
 * - palindrome: Struct para verificar se s[i..j] e palindromo em O(1).
 * - pal_begin(s): arr[i] = tamanho do maior palindromo comecando em i.
 * - pal_end(s):   arr[i] = tamanho do maior palindromo terminando em i.
 */

template<typename T>
vector<int> manacher(const T& s) {
    int l = 0, r = -1, n = s.size();
    vector<int> d1(n), d2(n);
    for (int i = 0; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i);
        while (i + k < n && i - k >= 0 && s[i + k] == s[i - k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        k++;
        while (i + k <= n && i - k >= 0 && s[i + k - 1] == s[i - k])
            k++;
        d2[i] = --k;
        if (i + k - 1 > r) l = i - k, r = i + k - 1;
    }
    vector<int> ret(2 * n - 1);
    for (int i = 0; i < n; i++) ret[2 * i] = 2 * d1[i] - 1;
    for (int i = 0; i < n - 1; i++) ret[2 * i + 1] = 2 * d2[i + 1];
    return ret;
}

template<typename T>
struct palindrome {
    vector<int> man;

    palindrome(const T& s) : man(manacher(s)) {}

    bool query(int i, int j) {
        return man[i + j] >= j - i + 1;
    }
};

template<typename T>
vector<int> pal_begin(const T& s) {
    int n = s.size() - 1;
    vector<int> ret(s.size());
    palindrome<T> p(s);
    ret[n] = 1;
    for (int i = n - 1; i >= 0; i--) {
        ret[i] = min(ret[i + 1] + 2, n - i + 1);
        while (!p.query(i, i + ret[i] - 1)) ret[i]--;
    }
    return ret;
}

template<typename T>
vector<int> pal_end(const T& s) {
    vector<int> ret(s.size());
    palindrome<T> p(s);
    ret[0] = 1;
    for (int i = 1; i < s.size(); i++) {
        ret[i] = min(ret[i - 1] + 2, i + 1);
        while (!p.query(i - ret[i] + 1, i)) ret[i]--;
    }
    return ret;
}
```

4.6 Suffix Array (NlogN)

```
/* Suffix Array O(N log N) — Prefix Doubling com Count Sort
 * sa[i]  = indice do i-esimo sufixo em ordem lexicografica.
 * rk[i]  = rank do sufixo s[i..] (inverso de sa).
 * lcp[i] = LCP entre sa[i-1] e sa[i] (lcp[0] = 0).
 * Adiciona '$' ao final da string internamente.
 * Complexidade: O(N log N) build, O(|P| log N) search.
 *
 * Aplicacoes:
 * - Busca de padrao P: search(p) retorna [lo, hi) no SA.
 * - Substring mais longa repetida: *max_element(lcp).
 * - Numero de substrings distintas: N*(N+1)/2 - sum(lcp).
 * - LCS de duas strings: SuffixArray::lcs(s, t).
 */

struct SuffixArray {
    string s;
    vector<int> sa, rk, lcp;

    SuffixArray() {}

    SuffixArray(string s) : s(s) {
        build();
        build_lcp();
    }

    void build() {
        s += '$';
        int n = s.size();
        sa.resize(n); rk.resize(n);

        vector<pair<char, int>> a(n);
        for (int i = 0; i < n; i++) a[i] = {s[i], i};
        sort(a.begin(), a.end());
        for (int i = 0; i < n; i++) sa[i] = a[i].second;
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            rk[sa[i]] = rk[sa[i - 1]] + (a[i].first != a[i - 1].first ? 1
: 0);

        for (int k = 0; (1 << k) < n; k++) {
            for (int i = 0; i < n; i++) sa[i] = (sa[i] - (1 << k) + n) % n;
            count_sort();
            vector<int> new_rk(n);
            new_rk[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                pair<int,int> prv = {rk[sa[i-1]], rk[(sa[i-1] + (1 << k)) % n]};
                pair<int,int> now = {rk[sa[i]],   rk[(sa[i]
+ (1 << k)) % n]};
                new_rk[sa[i]] = new_rk[sa[i - 1]] + (now != prv ? 1 : 0);
            }
            rk = new_rk;
        }
    }

    void count_sort() {
        int n = sa.size();
        vector<int> new_sa(n), cnt(n, 0), pos(n, 0);
        for (auto e : rk) cnt[e]++;
        for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
        for (auto e : sa) new_sa[pos[rk[e]]++] = e;
        sa = new_sa;
    }

    void build_lcp() {
        int n = sa.size();
        lcp.assign(n, 0);
        for (int i = 0, h = 0; i < n - 1; i++) {
            int j = sa[rk[i] - 1];
            while (s[i + h] == s[j + h]) h++;
            lcp[rk[i]] = h;
            if (h) h--;
        }
    }

    // Retorna [lo, hi) no SA onde p ocorre. hi - lo = n° de ocorrencias.
    pair<int,int> search(const string& p) const {
        int lo, hi, n = sa.size();
        {
            int l = 0, r = n;
            while (l < r) {
                int m = (l + r) / 2;
                if (s.compare(sa[m], p.size(), p) < 0) l = m + 1; else r = m;
            }
            lo = l;
        }
        {
            int l = 0, r = n;
            while (l < r) {
                int m = (l + r) / 2;
                if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1; else r = m;
            }
            hi = l;
        }
        return {lo, hi};
    }

    // LCS entre duas strings. Usa separador que nao aparece em nenhuma delas.
    static string lcs(const string& a, const string& b, char sep = '$') {
        SuffixArray suf(a + sep + b);
        int m = a.size(), best = 0, pos = 0;
        for (int i = 1; i < (int)suf.sa.size(); i++) {
            bool ga = suf.sa[i] < m, gb = suf.sa[i - 1] < m;
            if (ga != gb && suf.lcp[i] > best)
                { best = suf.lcp[i]; pos = suf.sa[i]; }
        }
        return suf.s.substr(pos, best);
    }
};
```

4.7 Suffix Array (Linear)

```
/* Suffix Array O(N) — SA-IS (Induced Sorting)
 * Complexidade: O(N) build. Use apenas se N for muito grande (> 10^6).
 * Mais complexo que o O(N log N) — prefira suffix-array-nlogn.hpp.
 *
 * Campos publicos apos construcao:
 *   sa[i]  = indice do i-esimo sufixo em ordem lexicografica
 *   rk[i]  = rank do sufixo s[i..] (inverso de sa)
 *   lcp[i] = tamanho do LCP entre sa[i-1] e sa[i]; lcp[0] = 0
 *
 * Metodos:
 *   search(p) → [lo, hi): intervalo no SA onde p ocorre; hi-lo = n° ocorrencias
 */

struct SuffixArrayLinear {
    string s;
    vector<int> sa, rk, lcp;

    SuffixArrayLinear() {}

    SuffixArrayLinear(const string& s) : s(s + '$') {
        vector<int> t(this->s.begin(), this->s.end());
        sa = sa_is(t, 256);
        int n = sa.size();
        rk.resize(n);
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        build_lcp();
    }

    pair<int,int> search(const string& p) const {
        int lo, hi, n = sa.size();
        {
            int l = 0, r = n;
            while (l < r) {
                int m = (l + r) / 2;
                if (s.compare(sa[m], p.size(), p) < 0) l = m + 1; else r = m;
```

```
            }
            lo = l;
        }
        {
            int l = 0, r = n;
            while (l < r) {
                int m = (l + r) / 2;
                if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1; else r = m;
            }
            hi = l;
        }
        return {lo, hi};
    }

    void build_lcp() {
        int n = sa.size();
        lcp.assign(n, 0);
        for (int i = 0, h = 0; i < n - 1; i++) {
            int j = sa[rk[i] - 1];
            while (s[i + h] == s[j + h]) h++;
            lcp[rk[i]] = h;
            if (h) h--;
        }
    }

    template<typename T>
    static vector<int> sa_is(const T& s, int sigma) {
        int n = s.size();
        if (n == 1) return {0};
        if (n == 2) return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};

        vector<bool> t(n);
        t[n - 1] = true;
        for (int i = n - 2; i >= 0; i--)
            t[i] = s[i] < s[i + 1] || (s[i] == s[i + 1] && t[i + 1]);

        auto is_lms = [&](int i) { return i > 0 && t[i] && !t[i - 1]; };

        vector<int> bkt(sigma + 1, 0);
        for (int c : s) bkt[c + 1]++;
        for (int i = 1; i <= sigma; i++) bkt[i] += bkt[i - 1];

        auto induced_sort = [&](const vector<int>& lms) {
            vector<int> sa(n, -1);
            vector<int> b = bkt;
            for (int i = (int)lms.size() - 1; i >= 0; i--)
                sa[--b[s[lms[i]] + 1]] = lms[i];
            b = bkt;
            sa[b[s[0]]++] = 0;
            for (int i = 0; i < n; i++) if (sa[i] > 0 && !t[sa[i] - 1])
                sa[b[s[sa[i] - 1]]++] = sa[i] - 1;
            b = bkt;
            for (int i = n - 1; i >= 0; i--) if (sa[i] > 0 && t[sa[i] - 1])
                    sa[--b[s[sa[i] - 1] + 1]] = sa[i] - 1;
            return sa;
        };

        vector<int> lms;
        for (int i = 0; i < n; i++) if (is_lms(i)) lms.push_back(i);
        vector<int> sa = induced_sort(lms);

        vector<int> rank_(n, -1);
        int cls = 0;
        for (int i = 0, prev = -1; i < n; i++) {
            if (!is_lms(sa[i])) continue;
            if (prev != -1) {
                bool diff = false;
                for (int d = 0; ; d++) {
                    if (s[prev+d] != s[sa[i]+d] || t[prev+d] != t[sa[i]+d]) {
                        diff = true; break;
                    }
                    if (d > 0 && (is_lms(prev+d) || is_lms(sa[i]+d))) break;
                }
                if (diff) cls++;
            }
            rank_[sa[i]] = cls;
            prev = sa[i];
        }

        vector<int> lms2, rank2;
        for (int i = 0; i < n; i++)
            if (rank_[i] != -1) {
                lms2.push_back(i); rank2.push_back(rank_[i]);
            }

        vector<int> sa2 = sa_is(rank2, cls);
        vector<int> sorted_lms(lms2.size());
        for (int i = 0; i < (int)sa2.size(); i++) sorted_lms[i] = lms2[sa2[i]];

        return induced_sort(sorted_lms);
    }
};
```

5 Geometry

5.1 Point

```
using ld = double;

const ld eps = 1e-9;
bool eq(ll a, ll b) { return a == b; }
bool eq(ld a, ld b) { return abs(a - b) <= eps; }

template<typename T>
struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}

    bool operator<(Point o)  const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(Point o) const { return eq(x, o.x) && eq(y, o.y); }
    bool operator!=(Point o) const { return !(*this == o); }

    Point operator+(Point o) const { return {x + o.x, y + o.y}; }
    Point operator-(Point o) const { return {x - o.x, y - o.y}; }
    Point operator*(T k)     const { return {x * k, y * k}; }
    Point operator/(T k)     const { return {x / k, y / k}; }

    T dot(Point o)   const { return x * o.x + y * o.y; }
    T cross(Point o) const { return x * o.y - y * o.x; }
  // area com sinal do triangulo (this, a, b)
    T cross(Point a, Point b) const { return (a - *this).cross(b - *this); }
    T dist2() const { return x * x + y * y; }

    ld len()         const { return hypot((ld)x, (ld)y); }
    Point<ld> norm() const { ld l = len(); return {x / l, y / l}; }
    Point perp()     const { return {-y, x}; }  // rotacao 90° CCW
    Point<ld> rotate(ld a) const { // a em radianos
        return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)};
    }

    friend ostream& operator<<(ostream& os, Point p) {
        return os << p.x << " " << p.y;
    }
};

using Pt  = Point<ll>;
using Ptd = Point<ld>;

// angulo entre os vetores p e q
template<typename T>
ld angle(Point<T> p, Point<T> q) { return atan2((ld)p.cross(q), (ld)p.dot(q)); }

// area com sinal do triangulo (p, q, r) — positivo = CCW
template<typename T>
T sarea(Point<T> p, Point<T> q, Point<T> r) { return p.cross(q, r); }

// se p, q, r sao colineares
template<typename T>
bool is_collinear(Point<T> p, Point<T> q, Point<T> r) {
    return eq(sarea(p, q, r), (T)0);
}
```

5.2 Line

```
#include "point.hpp"

struct Line {
    Ptd p, q;
    Line() {}
    Line(Ptd p, Ptd q) : p(p), q(q) {}
};

// se p pertence ao segmento r
bool isinseg(Ptd p, Line r) {
    Ptd a = r.p - p, b = r.q - p;
    return eq(a.cross(b), 0) && a.dot(b) < eps;
}

// projecao do ponto p na reta r
Ptd proj(Ptd p, Line r) {
    if (r.p == r.q) return r.p;
    Ptd v = r.q - r.p, u = p - r.p;
    return r.p + v * (u.dot(v) / v.dot(v));
}

// intersecao das retas r e s (retorna {INF,INF} se paralelas)
Ptd inter(Line r, Line s) {
    Ptd u = r.q - r.p, v = s.q - s.p, w = s.p - r.p;
    ld d = u.cross(v);
    if (eq(d, 0)) return {1e18, 1e18};
    return r.p + u * (w.cross(v) / d);
}

// se o segmento r intersecta o segmento s
bool interseg(Line r, Line s) {
    if (isinseg(r.p, s) || isinseg(r.q, s) ||
        isinseg(s.p, r) || isinseg(s.q, r)) return true;
    auto ccw = [](Ptd a, Ptd b, Ptd c) {
        return (b - a).cross(c - a) > eps;
    };
    return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
           ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}

// distancia do ponto p a reta r
ld disttoline(Ptd p, Line r) {
    return abs((r.q - r.p).cross(p - r.p)) / (r.q - r.p).len();
}

// distancia do ponto p ao segmento r
ld disttoseg(Ptd p, Line r) {
    if ((r.q - r.p).dot(p - r.p) < 0) return (p - r.p).len();
    if ((r.p - r.q).dot(p - r.q) < 0) return (p - r.q).len();
    return disttoline(p, r);
}

// distancia entre dois segmentos
ld distseg(Line a, Line b) {
    if (interseg(a, b)) return 0;
    return min({disttoseg(a.p, b), disttoseg(a.q, b),
                disttoseg(b.p, a), disttoseg(b.q, a)});
}
```

5.3 Circle

```
#include "line.hpp"

// centro da circunferencia que passa por a, b, c
Ptd getcenter(Ptd a, Ptd b, Ptd c) {
    Ptd m1 = (a + b) / 2, m2 = (a + c) / 2;
    return inter(Line(m1, m1 + (b - a).perp()),
                 Line(m2, m2 + (c - a).perp()));
}

// intersecao da circunferencia (c, r) com a reta ab
vector<Ptd> circ_line_inter(Ptd a, Ptd b, Ptd c, ld r) {
    vector<Ptd> ret;
    Ptd u = b - a, v = a - c;
    ld A = u.dot(u), B = v.dot(u), C = v.dot(v) - r * r;
    ld D = B * B - A * C;
    if (D < -eps) return ret;
    ret.push_back(c + v + u * (-B + sqrt(D + eps)) / A);
    if (D > eps)
        ret.push_back(c + v + u * (-B - sqrt(D)) / A);
    return ret;
}

// intersecao das circunferencias (a, r) e (b, R)
vector<Ptd> circ_inter(Ptd a, Ptd b, ld r, ld R) {
    vector<Ptd> ret;
    ld d = (b - a).len();
    if (d > r + R + eps || d + min(r, R) < max(r, R) - eps) return ret;
    ld x = (d * d - R * R + r * r) / (2 * d);
    ld y = sqrt(max(0.0, r * r - x * x));
    Ptd v = (b - a) / d;
    ret.push_back(a + v * x + v.perp() * y);
    if (y > eps)
        ret.push_back(a + v * x - v.perp() * y);
    return ret;
}
```

5.4 Polygon

```
#include "line.hpp"

// 2x a area com sinal (>0 = CCW)
template<typename T>
T area2(const vector<Point<T>>& poly) {
    T a = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++)
        a += poly[i].cross(poly[(i + 1) % n]);
    return a;
}

// area do poligono
template<typename T>
ld polarea(const vector<Point<T>>& poly) {
    return abs((ld)area2(poly)) / 2.0;
}


// 0 = fora, 1 = dentro, 2 = na borda
// O(n), funciona para qualquer poligono simples (convexo ou concavo)
template<typename T>
int winding(const vector<Point<T>>& poly, Point<T> p) {
    int n = poly.size(), w = 0;
    for (int i = 0; i < n; i++) {
        Point<T> a = poly[i] - p, b = poly[(i+1)%n] - p;
        if (a.y <= 0 && b.y > 0) {
            T c = a.cross(b);
            if (c > 0) w++;
            else if (c == 0) return 2;
        } else if (b.y <= 0 && a.y > 0) {
            T c = a.cross(b);
            if (c < 0) w--;
            else if (c == 0) return 2;
        } else if (b.y == 0 && a.y == 0) {
            if (min(a.x, b.x) <= 0 && 0 <= max(a.x, b.x)) return 2;
        }
    }
    return w != 0 ? 1 : 0;
}

// ponto no interior do casco convexo (CCW, sem colineares), O(log n)
template<typename T>
bool inside_hull(const vector<Point<T>>& hull, Point<T> p) {
    int n = hull.size();
    if (n == 1) return p == hull[0];
    if (n == 2) return hull[0].cross(hull[1], p) == 0
                    && (p - hull[0]).dot(hull[1] - hull[0]) >= 0
                    && (p - hull[1]).dot(hull[0] - hull[1]) >= 0;
    if (hull[0].cross(hull[1], p) <= 0 ||
        hull[0].cross(hull[n-1], p) >= 0) return false;
    int lo = 1, hi = n - 1;
    while (lo + 1 < hi) {
        int mid = (lo + hi) / 2;
        if (hull[0].cross(hull[mid], p) > 0) lo = mid;
        else hi = mid;
    }
    return hull[lo].cross(hull[lo+1], p) >= 0;
}

// corta o poligono com a reta r, mantendo pontos p tais que (r.p, r.q, p) e CCW
vector<Ptd> cut_polygon(vector<Ptd> poly, Line r) {
    vector<Ptd> ret;
    int n = poly.size();
    auto ccw = [&](Ptd a, Ptd b, Ptd c) { return (b - a).cross(c - a) > eps; };
    for (int i = 0; i < n; i++) {
        if (ccw(r.p, r.q, poly[i])) ret.push_back(poly[i]);
        if (n == 1) continue;
        Line s(poly[i], poly[(i + 1) % n]);
        Ptd p = inter(r, s);
        if (isinseg(p, s)) ret.push_back(p);
    }
    ret.erase(unique(ret.begin(), ret.end()), ret.end());
    if (ret.size() > 1 && ret.back() == ret[0]) ret.pop_back();
    return ret;
}

// se dois poligonos se intersectam — O(n*m)
bool interpol(const vector<Ptd>& v1, const vector<Ptd>& v2) {
    int n = v1.size(), m = v2.size();
    for (auto& p : v1) if (winding(v2, p)) return true;
    for (auto& p : v2) if (winding(v1, p)) return true;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (interseg(Line(v1[i], v1[(i+1)%n]),
                         Line(v2[j], v2[(j+1)%m]))) return true;
    return false;
}

// distancia entre dois poligonos
ld distpol(const vector<Ptd>& v1, const vector<Ptd>& v2) {
    if (interpol(v1, v2)) return 0;
    int n = v1.size(), m = v2.size();
    ld ret = 1e18;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            ret = min(ret, distseg(Line(v1[i], v1[(i+1)%n]),
                                   Line(v2[j], v2[(j+1)%m])));
    return ret;
}
```

5.5 Convex Hull

```
#include "polygon.hpp"

// Casco convexo CCW, O(N log N). Colineares sao descartados.
template<typename T>
vector<Point<T>> convex_hull(vector<Point<T>> pts) {
    int n = pts.size();
    if (n < 2) return pts;
    sort(pts.begin(), pts.end());
    vector<Point<T>> h;
    h.reserve(2 * n);
    for (int phase = 0; phase < 2; phase++) {
        int start = h.size();
        for (auto& p : pts) {
            while ((int)h.size() >= start + 2) {
                auto a = h[h.size()-2], b = h.back();
                if ((b - a).cross(p - a) > 0) break;
                h.pop_back();
            }
            h.push_back(p);
        }
        h.pop_back();
        reverse(pts.begin(), pts.end());
    }
    if (h.size() == 2 && h[0] == h[1]) h.pop_back();
    return h;
}

// Struct para operacoes O(log n) em casco convexo (CCW, sem colineares).
// Construir com convex_hull() antes.
struct ConvexPol {
    vector<Ptd> pol;

    ConvexPol() {}
    ConvexPol(vector<Ptd> v) : pol(convex_hull(v)) {}

    // se o ponto esta dentro do casco — O(log n)
    bool is_inside(Ptd p) {
        int n = pol.size();
        if (n == 0) return false;
        return inside_hull(pol, p);
    }

    // ponto extremo: cmp(p, q) = true se p e mais extremo que q
    int extreme(const function<bool(Ptd, Ptd)>& cmp) {
        int n = pol.size();
        auto extr = [&](int i, bool& cur_dir) {
            cur_dir = cmp(pol[(i+1)%n], pol[i]);
            return !cur_dir && !cmp(pol[(i+n-1)%n], pol[i]);
        };
        bool last_dir, cur_dir;
        if (extr(0, last_dir)) return 0;
        int l = 0, r = n;
        while (l + 1 < r) {
            int m = (l + r) / 2;
            if (extr(m, cur_dir)) return m;
            bool rel_dir = cmp(pol[m], pol[l]);
            if ((!last_dir && cur_dir) ||
                    (last_dir == cur_dir && rel_dir == cur_dir)) {
                l = m;
                last_dir = cur_dir;
            } else r = m;
        }
        return l;
    }

    // indice do ponto com maior produto interno com v — O(log n)
    int max_dot(Ptd v) {
        return extreme([&](Ptd p, Ptd q) { return p.dot(v) > q.dot(v); });
    }

    // tangentes a partir de p (indices {esquerda, direita}) — O(log n)
    pair<int,int> tangents(Ptd p) {
        auto L = [&](Ptd q, Ptd r) {
            return (r - p).cross(q - p) > eps;
        };
        auto R = [&](Ptd q, Ptd r) {
            return (q - p).cross(r - p) > eps;
```

```
2E2         };
FA8         return {extreme(L), extreme(R)};
00E     }
C9B };
```

5.6 Closest Pair

```
C97 #include "point.hpp"

 /* Closest Pair — Par de pontos mais proximos
  * Complexidade: O(N log N)
  *
  * closest_pair(pts) → {i, j} indices do par mais proximo (i < j)
  *
  * Distancia ao quadrado: (pts[i] - pts[j]).dist2()
  * Funciona com coordenadas inteiras (sem precisao flutuante).
  */

67A template<typename T>
CCC pair<int,int> closest_pair(const vector<Point<T>>& pts) {
CA0     int n = pts.size();
135     vector<int> idx(n);
295     iota(idx.begin(), idx.end(), 0);
658     sort(idx.begin(), idx.end(),
866          [&](int a, int b) { return pts[a].x < pts[b].x; });

3C9     T best = (pts[idx[0]] - pts[idx[1]]).dist2();
D94     pair<int,int> ans = {idx[0], idx[1]};

    // set ordenado por (y, indice)
3B1     set<pair<T,int>> active;
637     for (int l = 0, r = 0; r < n; r++) {
FAF         T delta = (T)ceil(sqrt((double)best)) + 1;
E85         while (pts[idx[r]].x - pts[idx[l]].x > delta) {
9CB             active.erase({pts[idx[l]].y, idx[l]});
63B             l++;
34B             delta = (T)ceil(sqrt((double)best)) + 1;
563         }
98F         auto lo = active.lower_bound({pts[idx[r]].y - delta, INT_MIN});
388         auto hi = active.upper_bound({pts[idx[r]].y + delta, INT_MAX});
DC0         for (auto it = lo; it != hi; it++) {
E0A             T d = (pts[idx[r]] - pts[it->second]).dist2();
C22             if (d < best) { best = d; ans = {idx[r], it->second}; }
A2C         }
0C6         active.insert({pts[idx[r]].y, idx[r]});
F7D     }
591     if (ans.first > ans.second) swap(ans.first, ans.second);
BA7     return ans;
844 }
```

5.7 Lattice

```
 // Teorema de Pick: pontos inteiros interiores a um poligono.
 // area2    = 2x a area com sinal (de area2())
 // boundary = Σ gcd(|dx|, |dy|) sobre as arestas
B9D long long lattice_points(long long area2, long long boundary) {
4EE     return (area2 - boundary + 2) / 2;
32D }
```

6 Others

6.1 Template

```
FA9 void solve() {}

E8D int main() {
886     ios::sync_with_stdio(false);
C5A     cin.tie(0); cout.tie(0);

ABE     int tc; cin >> tc; while (tc--) solve();

BB3     return 0;
10E }
```

6.2 Gen

```
42A #define endl '\n'


C8A mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
 // int x = rng();

463 int uniform(int l, int r) {
A7F     uniform_int_distribution<int> uid(l, r);
F54     return uid(rng);
D9E }

E8D int main() {
886     ios::sync_with_stdio(false);
B95     cin.tie(0);

BB3     return 0;
268 }
```

6.3 Checker

```
F3B int main(int argc, char* argv[]) {
4DE     ifstream in(argv[1]), out_sol(argv[2]), out_brt(argv[3]);

BB3     return 0; // return 0 for AC, 1 for WA
231 }
```

6.4 Stress Test

```
P=code   #codigo a testar
Q=brute  #brute correto
C=      #checker (vazio=cmp, ou ex: C=checker)
g++ ${P}.cpp -o sol -O2 || exit 1
g++ ${Q}.cpp -o brt -O2 || exit 1
g++ gen.cpp  -o gen -O2 || exit 1
[ -n "$C" ] && { g++ ${C}.cpp -o chk -O2 || exit 1; }
for ((i = 1; ; i++)) do
  echo $i
  ./gen $i > in
  ./sol < in > out
  ./brt < in > out2
  [ -n "$C" ] && ./chk in out out2 || cmp -s out out2
  if [ $? -ne 0 ]; then
    echo "--> entrada:"
    cat in
    echo "--> saida code:"
    cat out
    echo "--> saida brute:"
    cat out2
    break
  fi
done
```

7 Extra

7.1 Hash Function

```
 /* Lib Hash
  * Gera hash de trechos de codigo.
  * Ignora comentarios e espacos em branco.
  * O hash de uma linha com } e o hash de todo o codigo desde a { que a abriu.
  * Uso:
  *   g++ hash.cpp -o hash
  *   ./hash < code.cpp
  */

DE3 string getHash(string s){
E7F   ofstream("z.cpp") << s;
410   system("g++ -E -P -dD -fpreprocessed ./z.cpp | tr -d '[:space:]' | md5sum > sh");
F24   ifstream("sh") >> s;
A15   return s.substr(0, 3);
89F }
E8D int main(){
973   string l, t;
C3E   stack<string> st({""});
C61   while(getline(cin, l)){
54F         t = l;
242     for(auto c : l)
A53       if(c == '{') st.push(""); else
733       if(c == '}') t = st.top()+l, st.pop();
1B9     cout << getHash(t) + " " + l << endl;
DB4     st.top() += t;
C14   }
692 }
```